PROJETO PJBL — ARQUITETURA COMPLETA · ENTREGA 3

Conecta Store

Gestão de produtos, estoque e pedidos em uma arquitetura distribuída com microfrontend, API Gateway, BFF, microserviços e Azure Functions.

ARC42 | C4 Model | Canvas | Prints e evidências | Links da entrega

ALUNOS

Arthur Ambrozewicz Cidral

Erick Marlon Barbieri da Silva

Luan Alberti Estevinho

Vinícius de Oliveira Garcia

12 de novembro de 2026

Links da entrega

Tudo o que foi publicado está nesta página. Os mesmos links aparecem no README de cada repositório.

Aplicação na nuvem

O quê	Endereço
Frontend (microfrontend)	https://devinigarcia.github.io/pjbl-microfrontend
API Gateway (Azure API Management)	https://apim-pjbl-0a42b2.azure-api.net/conecta exige o cabeçalho `Ocp-Apim-Subscription-Key`; sem ele responde 401
Vídeo no YouTube (não listado)	pendente: URL_VIDEO

Swagger e documentação das APIs

Serviço	Endereço
Swagger do **BFF**	https://bff.agreeablesand-8dc3837e.brazilsouth.azurecontainerapps.io/docs
Swagger do **ms-produtos** (MongoDB)	https://ms-produtos.agreeablesand-8dc3837e.brazilsouth.azurecontainerapps.io/docs
Swagger do **ms-pedidos** (Azure SQL)	https://ms-pedidos.agreeablesand-8dc3837e.brazilsouth.azurecontainerapps.io/docs
Eventos (AsyncAPI — "Swagger de eventos")	https://ms-pedidos.agreeablesand-8dc3837e.brazilsouth.azurecontainerapps.io/events/docs
Swagger da **Azure Function**	https://func-pjbl-0a42b2.azurewebsites.net/api/docs

Repositórios no GitHub (públicos)

Projeto	Repositório
Microfrontend	https://github.com/devinigarcia/pjbl-microfrontend
BFF	https://github.com/devinigarcia/pjbl-bff
Microserviço Mongo (produtos)	https://github.com/devinigarcia/pjbl-ms-produtos
Microserviço SQL (pedidos)	https://github.com/devinigarcia/pjbl-ms-pedidos
Azure Function	https://github.com/devinigarcia/pjbl-function
Arquitetura (este documento, gateway, deploy)	https://github.com/devinigarcia/pjbl-arquitetura

Imagens no Docker Hub

Serviço	Imagem
BFF	https://hub.docker.com/r/vinigarciaa/pjbl-bff `(vinigarciaa/pjbl-bff:v1)`
ms-produtos	https://hub.docker.com/r/vinigarciaa/pjbl-ms-produtos `(vinigarciaa/pjbl-ms-produtos:v1)`

ms-pedidos	https://hub.docker.com/r/vinigarciaa/pjbl-ms-pedidos (`vinigarciaa/pjbl-ms-pedidos:v1`)

> O enunciado dá como exemplo o nome `dockerhubuser/pjbl/bff:v1`. O Docker Hub não aceita repositório aninhado (`usuario/pasta/imagem`), então usamos o nome do projeto como prefixo: `usuario/pjbl-bff:v1`. A Azure Function e o front não viram imagem: a Function roda no plano Flex Consumption e o front é um site estático.

Sumário

Parte I — ARC42

1. Introdução e objetivos
2. Restrições
3. Contexto e escopo
4. Estratégia de solução
5. Visão de blocos de construção
6. Visão de runtime
7. Visão de implantação
8. Conceitos transversais
9. Decisões arquiteturais
10. Requisitos de qualidade
11. Riscos e dívidas técnicas
12. Glossário

Parte II — C4 Model

- Nível 1: Contexto
- Nível 2: Containers
- Nível 3: Componentes (BFF, ms-pedidos, ms-produtos, Function)

Parte III — Canvas do projeto

Parte IV — Prints e evidências

- Swagger do BFF e dos microserviços
- Eventos (AsyncAPI)
- Frontend funcionando
- API Gateway
- Docker Hub, GitHub Actions e testes

Parte V — Conferência com o enunciado

Parte I — ARC42 · 1. Introdução e objetivos

1.1 Visão geral

O **Conecta Store** é um sistema de retaguarda para uma loja pequena: cadastro de produtos, controle de estoque por movimentações e registro de pedidos de clientes, com um painel que resume a operação. O domínio é simples de propósito. O objetivo do projeto não é o negócio em si, e sim **demonstrar uma arquitetura distribuída completa funcionando na nuvem**: microfrontend, API Gateway, BFF, dois microserviços com bancos diferentes e uma Azure Function, ligados por chamadas síncronas e por eventos.

Escolhemos produtos e pedidos porque os dois domínios precisam um do outro de um jeito que justifica cada peça: o pedido depende do preço e do estoque do produto (o que dá trabalho de verdade ao BFF) e a venda precisa baixar o estoque (o que dá um motivo real para usar eventos em vez de uma chamada direta entre microserviços).

1.2 Requisitos principais

#	Requisito do enunciado	Como foi atendido
R1	Microfrontend (React ou Angular) que consome só o BFF	React + Vite com Module Federation: um *shell* e dois *remotes* (Produtos e Pedidos). O único endereço que o front conhece é o do API Gateway.
R2	BFF em Node.js com agregação, proxy de CRUD e chamada à Function	Node + Express. `GET /aggregated-data` consulta os dois microserviços em paralelo e a Function, e devolve um JSON só. Proxy do CRUD e criação de pedido orquestrada.
R3	Microserviço 1 com MongoDB Atlas, CRUD e Swagger	**ms-produtos**: produtos e movimentações de estoque no Atlas (M0). Swagger em `/docs`.
R4	Microserviço 2 com Azure SQL, CRUD e Swagger	**ms-pedidos**: pedidos, itens e outbox no Azure SQL (oferta gratuita). Swagger em `/docs`.
R5	Azure Function com HTTP Trigger / Message Trigger	Os dois: HTTP Trigger que calcula indicadores e Queue Trigger que consome eventos de pedido.
R6	API Gateway com roteamento, segurança e entrada central	Azure API Management (Consumption): subscription key, rate limit, CORS e segredo que impede acesso direto ao BFF.
R7	Event-Driven Architecture	Transactional Outbox no ms-pedidos → Azure Storage Queue → Function → movimentação de estoque no ms-produtos, com idempotência.
R8	Clean Architecture e Vertical Slice	Domain / Application / Infrastructure / API em todos os backends; uma pasta por caso de uso (`CreateOrder`, `GetOrder`...).
R9	Testes unitários e de arquitetura	659 testes automatizados somando os cinco projetos, incluindo testes de arquitetura em todos eles.
R10	Docker Hub, GitHub público, README, vídeo	Imagens publicadas pelo GitHub Actions; seis repositórios públicos com README; vídeo não listado no YouTube.

1.3 Metas de qualidade

#	Meta	O que significa aqui
Q1	**Manutenibilidade**	Regras de dependência entre camadas verificadas por teste. Criar uma funcionalidade nova não exige mexer nas que existem.
Q2	**Resiliência**	Se um serviço cair, o painel continua mostrando o que os outros responderam. Nenhum evento de pedido se perde.
Q3	**Consistência dos dados**	Estoque nunca fica negativo e uma mensagem repetida não baixa o estoque duas vezes.
Q4	**Segurança**	Uma única porta de entrada. Nenhum segredo no código ou nos repositórios.
Q5	**Observabilidade**	Dá para seguir uma requisição do navegador até o banco pelo `x-correlation-id`.
Q6	**Custo zero**	Tudo roda em camadas gratuitas ou de consumo.

1.4 Partes interessadas

Quem	O que espera
Professor / avaliador	Ver todos os itens do enunciado funcionando em URLs públicas, com documentação e código organizados.
Equipe (Arthur Ambrozewicz Cidral, Erick Marlon Barbieri da Silva, Luan Alberti Estevinho, Vinícius de Oliveira Garcia)	Um projeto que dê para entender, testar e publicar sem depender de máquina específica.
Operador da loja (usuário)	Cadastrar produtos, lançar pedidos e ver o estoque atualizado sem precisar saber o que acontece por trás.

2. Restrições

Tipo	Restrição	Consequência
Técnicas (enunciado)	BFF em Node.js	Usamos TypeScript/Node em todos os projetos, para ter uma linguagem só.
	MongoDB Atlas (free) e Azure SQL (free)	Um banco por microserviço, sem compartilhamento. O banco SQL gratuito "dorme" após 15 min sem uso.
	Azure Function com HTTP e/ou mensagem	Storage Queue como mensageria (já vem com a Function, sem custo).
	Imagens no Docker Hub	Serviços empacotados em Docker; a plataforma precisa rodar imagem pública.
	Demonstração só com URLs da nuvem	Tudo precisa estar publicado; nada de *localhost* no vídeo.

Organizacionais	Sem orçamento	Só camadas gratuitas e de consumo (Azure for Students, Atlas M0, GitHub, Docker Hub).
	Equipe pequena e prazo de disciplina	Domínio enxuto; esforço concentrado na arquitetura, nos testes e na documentação.
	Repositórios públicos	Nenhum segredo pode ir para o Git. Configuração só por variável de ambiente.
Convenções	Idioma	Código, rotas e campos em inglês; interface, documentação e comentários em português.
	Estrutura	Mesma organização de pastas e os mesmos testes de arquitetura em todos os backends.

3. Contexto e escopo

3.1 Contexto de negócio

O operador da loja usa o sistema pelo navegador. O sistema, por sua vez, depende de três serviços externos: o MongoDB Atlas (banco de produtos), o GitHub (código, automações e hospedagem do front) e o Docker Hub (imagens). O diagrama de contexto está na Parte II (C4 nível 1).

Parceiro	Entrada	Saída
Operador da loja	Cadastro de produtos, entradas de estoque, pedidos, mudanças de status	Listas, painel de indicadores, alertas de estoque, histórico de eventos
MongoDB Atlas	Consultas e gravações de produtos e movimentações	Documentos
GitHub	`git push` da equipe	Resultado dos testes, imagens publicadas, site do front no ar
Docker Hub	Imagens enviadas pelo GitHub Actions	Imagens baixadas pelo Azure Container Apps

3.2 Contexto técnico

Interface	Protocolo	Observação
Navegador → Gateway	HTTPS + JSON	Cabeçalho `Ocp-Apim-Subscription-Key`; CORS liberado pelo gateway
Gateway → BFF	HTTPS + JSON	Gateway injeta `x-gateway-secret`; sem ele o BFF responde 401
BFF → microserviços	HTTPS + JSON (REST)	`x-api-key` interna e `x-correlation-id`
BFF → Function	HTTPS (HTTP Trigger)	`x-functions-key`
ms-pedidos → fila	Azure Storage Queue	Mensagem JSON em Base64; contrato em AsyncAPI
Fila → Function	Queue Trigger	Até 5 tentativas; depois a mensagem vai para `order-events-poison`
Function → ms-produtos	HTTPS + JSON	`POST /products/{id}/stock-movements` com `eventId` (idempotência)
ms-produtos → Atlas	Driver MongoDB sobre TLS	—
ms-pedidos → Azure SQL	TDS criptografado (pacote `mssql`)	—

O contrato campo a campo de cada interface está em `docs/contratos.md` no repositório `pjbl-arquitetura`.

4. Estratégia de solução

Problema	Decisão	Por quê
O front precisa de dados de três lugares	BFF com `/aggregated-data`	Uma chamada só no navegador; o BFF paraleliza e tolera falha de uma das fontes.
Pedido precisa de preço e estoque do produto	Orquestração no BFF (`POST /orders`)	Os microserviços não se conhecem. O preço vem sempre do catálogo, nunca do navegador.
Venda precisa baixar o estoque	Evento `order.created` em vez de chamada direta	ms-pedidos funciona mesmo com o ms-produtos fora do ar; a baixa acontece quando ele voltar.
Gravar o pedido e publicar o evento sem perder nenhum dos dois	Transactional Outbox	Pedido e evento na mesma transação SQL. Um relay publica depois.
Fila entrega a mesma mensagem mais de uma vez	Idempotência por `eventId` no ms-produtos	Índice único no MongoDB; repetição responde 200 sem alterar o saldo.
Manter o código organizado em cinco projetos	Clean Architecture + Vertical Slice + testes de arquitetura	A regra vira teste automatizado, não combinado de equipe.
Ninguém deve pular o gateway	Segredo compartilhado gateway ↔ BFF	Mesmo sabendo a URL do BFF, a chamada direta dá 401.
Publicar sem depender da máquina de alguém	GitHub Actions + script de infraestrutura	Testes, imagens e deploy reproduzíveis.

5. Visão de blocos de construção

5.1 Nível 1 — o sistema por dentro

O diagrama está na Parte II (C4 nível 2 — Containers). Cada bloco é um repositório e uma unidade de implantação.

Bloco	Responsabilidade	Interfaces principais
Microfrontend	Interface. *Shell* com o painel e a navegação; *remote* de Produtos; *remote* de Pedidos. Conversam entre si por eventos do navegador (`CustomEvent`).	Consome o gateway. Mostra um painel "Chamadas ao BFF" com correlation-id e tempos.
API Gateway	Entrada única: autenticação por chave, limite de requisições, CORS, roteamento para o BFF.	`/conecta/*` → BFF
BFF	Agregação, orquestração da criação de pedido e proxy do CRUD. Não tem banco.	`GET /aggregated-data`, `POST /orders`, `/products`, `/orders`, `/events`
ms-produtos	Dono do catálogo e do estoque. O estoque só muda por movimentação.	`/products`, `/products/{id}/stock-movements`
ms-pedidos	Dono dos pedidos e da máquina de estados. Publica os eventos.	`/orders`, `/orders/{id}/status`, `/events`, AsyncAPI
Azure Function	Cálculo dos indicadores (HTTP) e aplicação dos eventos no estoque (fila).	`POST /api/indicators`, Queue Trigger `order-events`

5.2 Nível 2 — por dentro de cada serviço

Os quatro backends seguem a mesma organização. Os diagramas de componentes estão na Parte II (C4 nível 3).

```
src/
  domain/            entidades, regras e erros — TypeScript puro, nenhum pacote externo
  application/
    ports/           interfaces que a aplicação precisa (repositórios, gateways)
    features/        um diretório por caso de uso (vertical slice)
      CreateOrder/CreateOrder.ts        schema de validação + classe CreateOrderHandler
      GetOrder/GetOrder.ts              ...
  infrastructure/    implementações das portas: MongoDB, SQL, fila, clientes HTTP, config
  api/
    features/        adaptador HTTP fino de cada caso de uso
      CreateOrder/CreateOrderEndpoint.ts
    middlewares/     correlationId, apiKey, errorHandler
  main.ts            composition root — o único arquivo que conhece infrastructure e api
tests/
  unit/  integration/  architecture/
```

Serviço	Casos de uso (uma pasta cada)
BFF	`GetAggregatedData`, `PlaceOrder`, `ProductsProxy`, `OrdersProxy`, `CheckHealth`

ms-produtos	`CreateProduct`, `ListProducts`, `GetProduct`, `UpdateProduct`, `DeleteProduct`, `RegisterStockMovement`, `ListStockMovements`
ms-pedidos	`CreateOrder`, `ListOrders`, `GetOrder`, `UpdateOrder`, `ChangeOrderStatus`, `DeleteOrder`, `ListOrderEvents`, `DispatchOrderEvents`
Function	`CalculateIndicators`, `ProcessOrderEvent`

Regras de negócio que valem destacar

- **Produto:** SKU único, em maiúsculas; preço maior que zero; estoque nunca negativo; "estoque baixo" quando ≤ 5.
- **Estoque:** não é editável. Só muda por movimentação de ENTRADA ou SAÍDA, que fica no histórico com o saldo resultante.
- **Pedido:** de 1 a 20 itens, sem produto repetido, quantidade de 1 a 999; total calculado no servidor.
- **Status:** PENDENTE → PAGO ou CANCELADO; PAGO → ENVIADO ou CANCELADO; ENVIADO → ENTREGUE. ENTREGUE e CANCELADO são finais.
- **Edição:** dados do cliente só mudam com o pedido PENDENTE; itens nunca mudam.
- **Exclusão:** só pedido CANCELADO pode ser excluído.
- **Criação pelo BFF:** nome e preço vêm do catálogo; estoque insuficiente bloqueia o pedido (409) dizendo quanto foi pedido e quanto há.

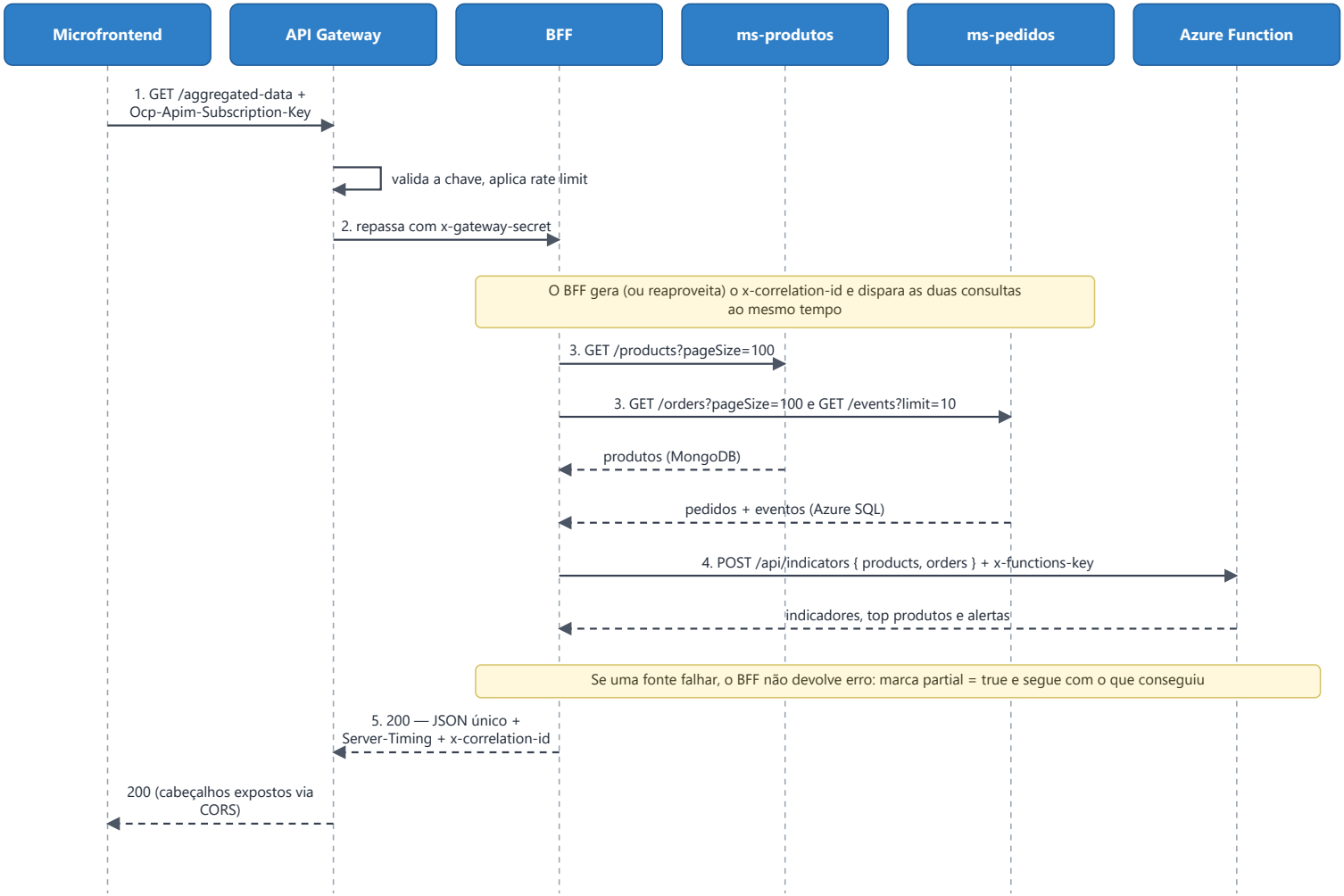
6. Visão de runtime

Dois cenários resumem o comportamento do sistema: a leitura agregada (síncrona) e a criação de pedido com baixa de estoque (síncrona até a resposta, assíncrona depois).

6.1 Cenário 1 — Painel: GET /aggregated-data

Visão de runtime 1 — GET /aggregated-data

O dashboard pede tudo de uma vez; o BFF consulta os dois microserviços em paralelo e depois a Function

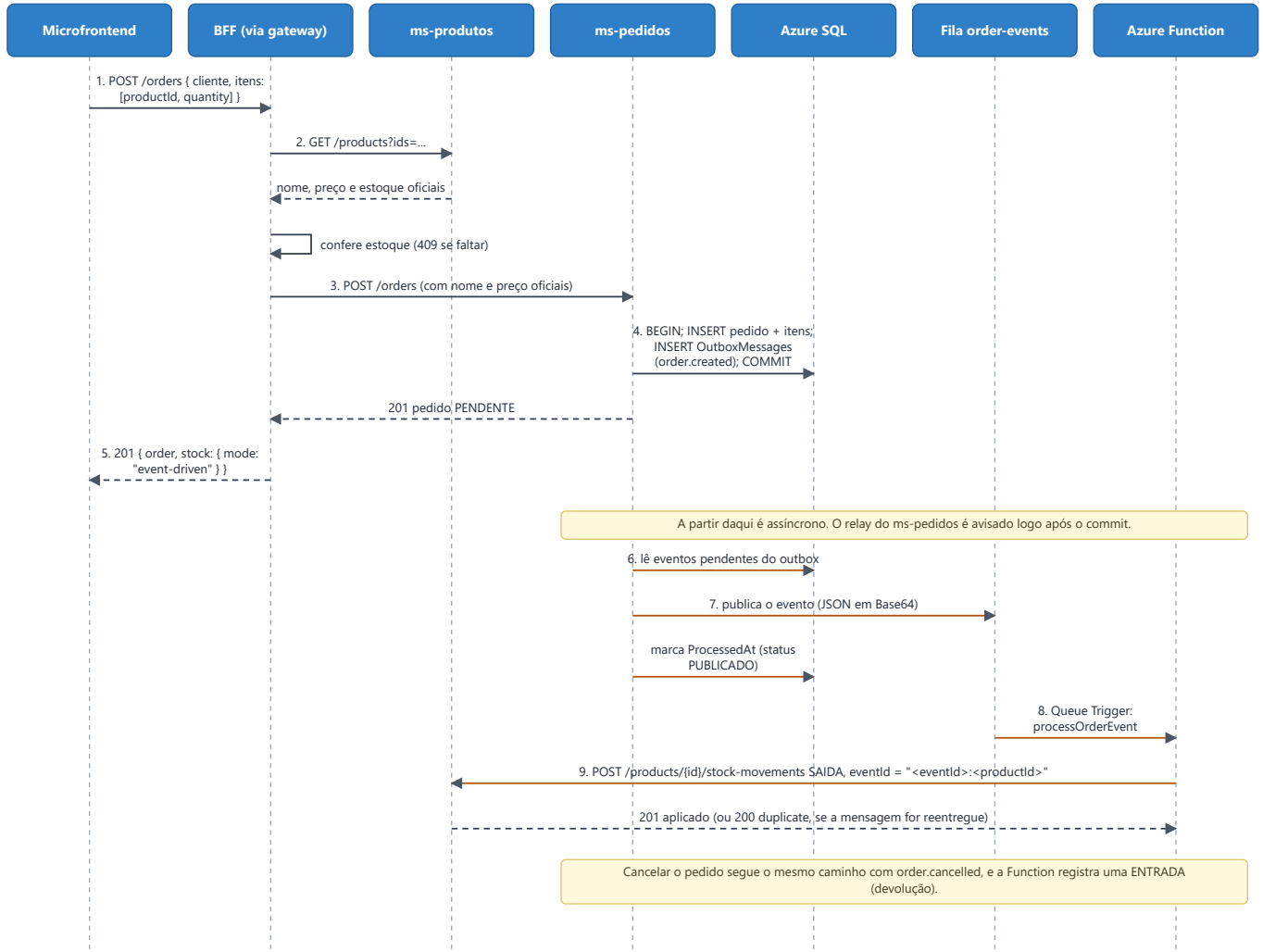


O BFF é o único que sabe quantas fontes existem. O front faz uma chamada e recebe um JSON.

6.2 Cenário 2 — Pedido criado e estoque baixado por evento

Visão de runtime 2 — Criação de pedido e baixa de estoque por evento

Parte síncrona (até o 201) e parte assíncrona (laranja): outbox → fila → Function → estoque



Passos 1 a 5 são síncronos; de 6 a 9 (em laranja) acontecem depois, sem o usuário esperar.

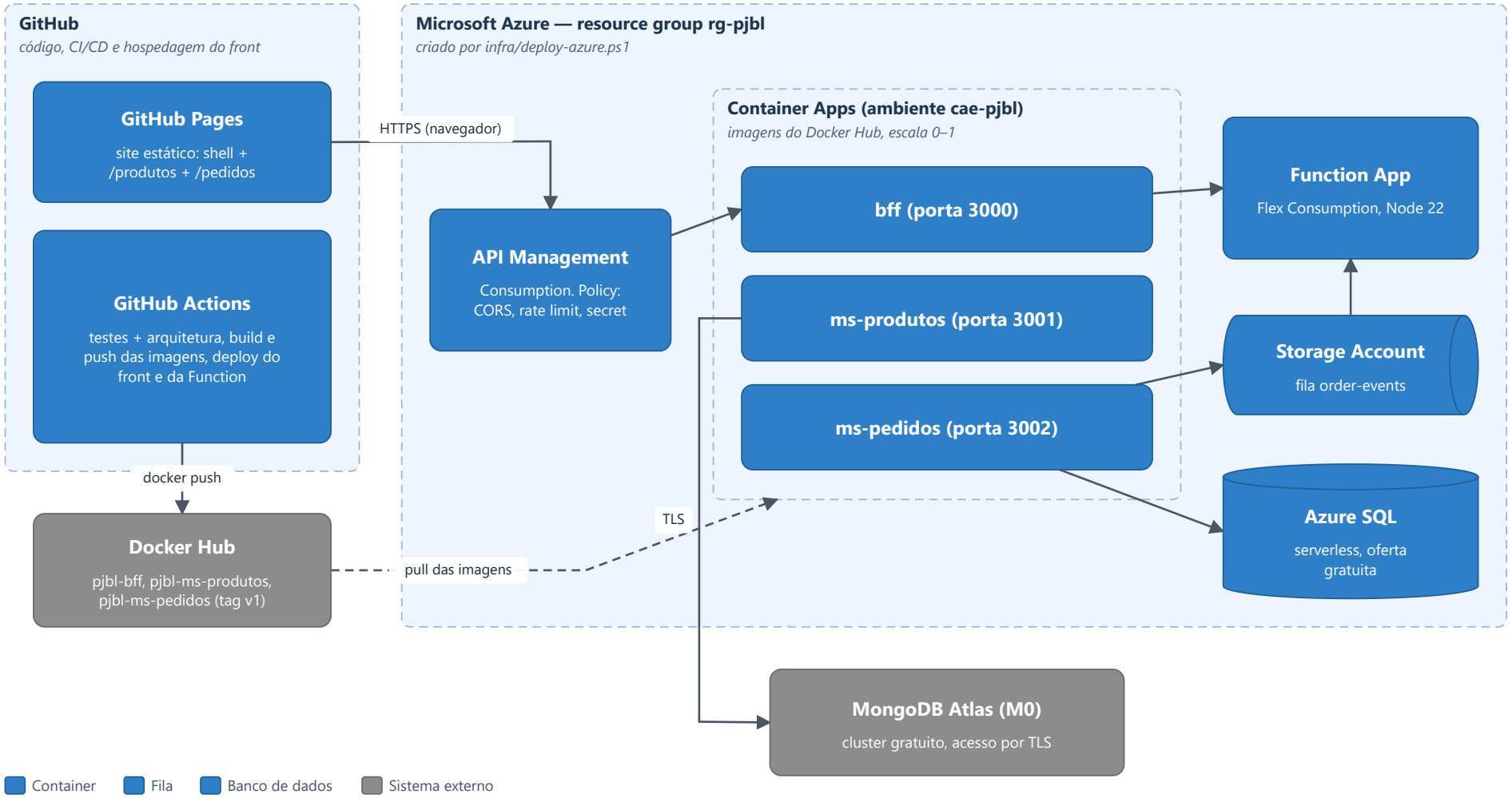
6.3 O que acontece quando algo falha

Situação	Comportamento
ms-produtos (ou ms-pedidos) fora do ar durante o `/aggregated-data`	Resposta 200 com `partial: true`, o bloco daquela fonte como `null` e o motivo em `sources`. O painel mostra o aviso e o restante dos dados.
Function fora do ar	Produtos e pedidos aparecem normalmente; os indicadores ficam indisponíveis.
Microserviço fora do ar numa operação de CRUD	BFF responde 502 `UPSTREAM_UNAVAILABLE` (504 em timeout), no formato padrão de erro.
Fila indisponível na hora de publicar	O pedido já está gravado. O evento fica PENDENTE/FALHOU no outbox e o relay tenta de novo. `GET /events` mostra a situação.
ms-produtos fora do ar quando a Function processa o evento	A Function relança o erro; a fila reentrega até 5 vezes e depois move para a fila *poison*.
Mesma mensagem entregue duas vezes	ms-produtos reconhece o `eventId` e responde 200 `duplicate: true`, sem mexer no saldo.
Dois cancelamentos simultâneos do mesmo pedido	Atualização condicional (`WHERE Status = @esperado`): só um vence; o outro recebe 409. Sai um único evento de devolução.
Banco SQL gratuito "dormindo"	O ms-pedidos tenta de novo por alguns segundos; se não der, responde 503 com `Retry-After` e o front tenta outra vez.

7. Visão de implantação

Visão de implantação

Onde cada parte roda na nuvem — tudo em camadas gratuitas ou de consumo



Toda a parte Azure é criada pelo script `infra/deploy-azure.ps1`, que pode ser executado de novo sem duplicar nada.

7.1 Onde cada coisa roda

Parte	Serviço de nuvem	Camada	Observação
Microfrontend	GitHub Pages	Gratuita	Site estático único: `/`, `/produtos/`, `/pedidos/`
API Gateway	Azure API Management	Consumption	1 milhão de chamadas/mês sem custo
BFF e microserviços	Azure Container Apps	Consumo	Imagens do Docker Hub; 0 a 1 réplica
Function	Azure Functions	Flex Consumption	Node 22; o plano Consumption em Linux está sendo aposentado
Fila	Azure Storage Queue	Uso mínimo	Mesma conta de armazenamento da Function
Banco de pedidos	Azure SQL Database	Oferta gratuita serverless	Pausa após 15 min sem uso; nunca gera cobrança (AutoPause)
Banco de produtos	MongoDB Atlas	M0 (gratuita)	—
Imagens	Docker Hub	Gratuita	Repositórios públicos

> O enunciado fala em "Azure SQL Database (Free 1 DTU)". Hoje a oferta gratuita da Azure para banco SQL é a *serverless* (100 mil vCore-segundos por mês), que é a que usamos. Para o projeto, o efeito é o mesmo: um Azure SQL gratuito.

7.2 Esteira de entrega

1. **Push na** `main` de qualquer repositório dispara o GitHub Actions.
2. **Job de testes:** checagem de tipos, testes unitários, de integração e de arquitetura, e build.
3. **Job de publicação:** nos serviços com Docker, build e *push* da imagem (`v1`, `latest`, `sha-…`); no front, deploy no GitHub Pages; na Function, deploy do pacote.
4. **Infraestrutura:** `infra/deploy-azure.ps1` cria ou atualiza os recursos, gera os segredos e aponta os Container Apps para as imagens.

8. Conceitos transversais

8.1 Clean Architecture e Vertical Slice, garantidas por teste

As dependências sempre apontam para dentro: **API** → **Application** → **Domain** e **Infrastructure** → **Application** → **Domain**. API e Infrastructure não se conhecem; quem liga as duas é o `main.ts`. Dentro de Application e API, o código é organizado por funcionalidade, e uma funcionalidade não importa outra.

Para isso não depender de disciplina, cada projeto tem testes de arquitetura próprios: um pequeno scanner lê os `import` de todos os arquivos e verifica as regras abaixo. Se alguém quebrar uma delas, a esteira falha.

Regra verificada	Exemplo de violação que o teste pega
Domain não importa nenhuma outra camada nem pacotes	`import express from 'express'` em `domain/`
Application só depende do Domain (pacotes: só `zod`)	caso de uso importando o repositório SQL concreto
API não importa Infrastructure, e vice-versa	endpoint lendo `infrastructure/config`
Driver de banco, SDK de fila e `fetch` só na Infrastructure	`mongodb` ou `mssql` importado num handler
Uma feature não importa outra	`CreateOrder` importando `GetOrder`
Toda feature tem `<Feature>Handler` e `<Feature>Endpoint`	pasta de feature sem adaptador HTTP

Os próprios testes plantam violações em arquivos fictícios e conferem que cada regra reclama — assim sabemos que um "passou" não é falso positivo. No front, o teste equivalente garante que os *remotes* não importam um do outro e que nenhuma tela usa `fetch` direto.

8.2 Event-driven: outbox, fila e idempotência

- **Transactional Outbox.** O ms-pedidos grava o pedido e o evento (tabela `OutboxMessages`) na mesma transação. Ou os dois existem, ou nenhum.
- **Relay.** Um componente do ms-pedidos lê os eventos pendentes e publica na fila. Ele é acionado logo depois do *commit* e só continua tentando enquanto houver evento pendente — não fica consultando o banco à toa, o que manteria o SQL gratuito acordado e gastaria a cota do mês.
- **Entrega pelo menos uma vez.** Uma mensagem pode chegar repetida; nunca se perde.
- **Consumidor idempotente.** Cada movimentação leva `eventId = "<id do evento>:<id do produto>"`. O ms-produtos tem índice único nesse campo.
- **Contrato documentado.** AsyncAPI 2.6 servido pelo ms-pedidos em `/events/docs`.
- **Consistência eventual assumida.** O estoque baixa alguns segundos depois do pedido. A resposta do BFF já avisa (`stock.mode: "event-driven"`) e o painel se atualiza sozinho.

8.3 Segurança

Camada	Controle

Borda	Subscription key obrigatória no gateway; limite de 120 chamadas/minuto por chave; CORS; HTTPS em tudo.
Gateway → BFF	Cabeçalho `x-gateway-secret` (Named Value secreto do APIM). O BFF compara em tempo constante e recusa quem não tem.
BFF → serviços	`x-api-key` própria de cada microserviço; `x-functions-key` na Function.
Aplicação	Validação de toda entrada (zod); consultas SQL parametrizadas; limite de tamanho do corpo; `helmet`; rate limit também no BFF; container sem usuário root.
Segredos	Só em variáveis de ambiente / secrets do Container Apps e do GitHub. Gerados pelo script de deploy, nunca impressos nem versionados. Cabeçalhos sensíveis mascarados nos logs.

> Limitação conhecida: a subscription key vai dentro do JavaScript do front, então qualquer pessoa consegue vê-la. Ela identifica a aplicação e sustenta o limite de uso, mas não substitui login de usuário — que ficou fora do escopo (ver seção 11).

8.4 Observabilidade

- **Correlation-id.** Toda requisição recebe um `x-correlation-id` (ou reaproveita o que veio). Ele segue do front ao BFF, aos microserviços, à Function e dentro do próprio evento.
- **Logs estruturados** em JSON (pino), com o correlation-id em cada linha.
- **Server-Timing.** O `/aggregated-data` informa o tempo de cada fonte, visível na aba *Network* do navegador e no painel "Chamadas ao BFF" do front.
- **Health checks.** `/health` em todos os serviços; o do BFF mostra a situação de cada dependência.
- **Eventos visíveis.** `GET /events` lista o outbox com status PUBLICADO, PENDENTE ou FALHOU.

8.5 Tratamento de erros

Todo erro, em qualquer serviço, tem o mesmo formato: `{ "error": { "code", "message", "details", "correlationId" } }`. As camadas internas só lançam exceções; um único *error handler* na API traduz para HTTP.

HTTP	Quando	Exemplos de `code`
400	Entrada inválida	`VALIDATION_ERROR` (com a lista de campos), `INVALID_JSON`
401	Sem credencial	`UNAUTHORIZED`
404	Recurso não existe	`PRODUCT_NOT_FOUND`, `ORDER_NOT_FOUND`
409	Conflito com o estado atual	`SKU_ALREADY_EXISTS`, `INSUFFICIENT_STOCK`, `INVALID_STATUS_TRANSITION`, `ORDER_NOT_DELETABLE`
422	Regra de negócio	`DUPLICATED_ITEM`, `PRODUCT_NOT_FOUND` (na criação do pedido)
502 / 504	Serviço de trás indisponível / lento	`UPSTREAM_UNAVAILABLE`, `UPSTREAM_TIMEOUT`

8.6 Testes

Projeto	Testes	O que cobrem
ms-produtos	161	Domínio, casos de uso, HTTP, arquitetura e integração com MongoDB real (inclusive 10 baixas concorrentes com estoque 5).
ms-pedidos	228	Máquina de estados, outbox, relay, publicação na fila, HTTP, arquitetura. Inclui 18 testes de integração com SQL Server real, que só rodam quando há um banco disponível.
Function	85	Cálculo dos indicadores, tradução de eventos, retry x descarte, cliente HTTP, arquitetura.
BFF	96	Agregação e falha parcial, pedido orquestrado, integração com servidores HTTP falsos (gateway secret, 502/504), arquitetura.
Microfrontend	89	Componentes (painel, formulários, pedido), cliente HTTP e arquitetura dos *remotes*.
Total	**659**	Rodam no GitHub Actions a cada *push* (os 18 de SQL real, sob demanda).

9. Decisões arquiteturais

ADR-01 — TypeScript e Node.js em todos os projetos

Contexto: o enunciado pede Node no BFF e deixa os demais livres. **Decisão:** uma linguagem só, do front à Function. **Consequências:** mesma estrutura, mesmas ferramentas de teste e o mesmo scanner de arquitetura em todos os repositórios; qualquer pessoa da equipe mexe em qualquer serviço. Abrimos mão de mostrar outra plataforma (.NET, por exemplo).

ADR-02 — Transactional Outbox para publicar eventos

Contexto: gravar no banco e publicar na fila são duas operações; se uma falhar, pedido e estoque divergem. **Decisão:** gravar o evento numa tabela na mesma transação do pedido e publicar depois, por um relay. **Consequências:** nenhum evento se perde; em troca, a entrega pode se repetir, o que exige consumidor idempotente (ADR-03).

ADR-03 — Idempotência no ms-produtos, com índice único

Contexto: a fila entrega pelo menos uma vez. **Decisão:** cada movimentação leva um `eventId` e o MongoDB tem índice único parcial nesse campo. **Consequências:** a garantia fica no banco, então vale até com duas instâncias processando ao mesmo tempo. A baixa usa `$inc` condicional (`stock ≥ quantidade`), sem precisar de transação.

ADR-04 — Azure Storage Queue em vez de Service Bus

Contexto: precisávamos de mensageria gratuita. **Decisão:** Storage Queue, que usa a conta de armazenamento que a Function já exige. **Consequências:** custo praticamente zero e gatilho nativo. Não tem tópicos nem ordenação garantida, o que não faz falta com um único consumidor; se surgirem mais consumidores, o ponto de troca é só a classe publicadora.

ADR-05 — Azure API Management em vez de AWS API Gateway

Contexto: o enunciado cita o gateway da AWS como opção. **Decisão:** APIM na camada Consumption. **Consequências:** uma nuvem só, uma conta só (sem cartão) e as rotas importadas do OpenAPI do BFF. A definição equivalente para AWS está versionada em `gateway/aws/`, caso se queira trocar.

ADR-06 — Azure Container Apps para BFF e microserviços

Contexto: o enunciado exige imagens no Docker Hub. **Decisão:** Container Apps puxando as imagens públicas. **Consequências:** HTTPS e URL pública prontos, escala até zero (sem custo parado). A primeira chamada depois de um tempo ocioso demora alguns segundos.

ADR-07 — Microfrontend com Module Federation num site só

Contexto: "microfrontend" precisa ser mais que uma SPA comum. **Decisão:** *shell* + dois *remotes* carregados em tempo de execução, publicados como um único site estático. **Consequências:** cada *remote* tem build próprio e também abre sozinho; se um falhar, o resto continua. Em contrapartida, os *remotes* precisam de build para rodar localmente.

ADR-08 — Validação de estoque no BFF, baixa por evento

Contexto: o pedido não deveria ser aceito sem estoque, mas a baixa é assíncrona. **Decisão:** o BFF confere o estoque antes de criar o pedido; a baixa real acontece pelo evento. **Consequências:** boa experiência no caso comum. Dois pedidos simultâneos do último item podem ser aceitos; o segundo evento recebe 409 do ms-produtos e fica registrado como alerta (ver risco R2).

10. Requisitos de qualidade

10.1 Árvore de qualidade

- **Manutenibilidade**
  - dependências entre camadas
  - isolamento entre funcionalidades
  - cobertura por testes
- **Confiabilidade**
  - tolerância a falha parcial
  - nenhum evento perdido
  - idempotência
- **Segurança**
  - entrada única
  - segredos fora do código
- **Observabilidade**
  - rastreio ponta a ponta
  - saúde por dependência
- **Desempenho**
  - consultas em paralelo no BFF
- **Custo**
  - camadas gratuitas

10.2 Cenários

#	Estímulo	Resposta esperada	Como verificamos
C1	Um desenvolvedor importa o driver do banco dentro de um caso de uso	A esteira falha apontando arquivo e import	Testes de arquitetura
C2	ms-pedidos cai enquanto o painel está aberto	Painel mostra produtos, avisa que pedidos estão indisponíveis; HTTP 200	Teste de integração do BFF
C3	10 baixas simultâneas num produto com 5 unidades	5 aceitas, 5 recusadas, saldo final 0	Teste de integração com MongoDB
C4	A fila reentrega o mesmo evento	Estoque baixa uma vez só	Teste do ms-produtos e da Function
C5	Alguém chama o BFF direto, sem passar pelo gateway	401 `UNAUTHORIZED`	Teste de integração do BFF; print na Parte IV
C6	Front tenta criar pedido com preço alterado	Preço enviado é ignorado; vale o do catálogo	Teste unitário do `PlaceOrder`
C7	Usuário relata erro numa tela	Com o correlation-id mostrado na tela acham-se os logs de todos os serviços	Demonstração no vídeo

11. Riscos e dívidas técnicas

#	Risco / dívida	Impacto	O que fizemos ou faríamos
R1	Cold start: containers em escala zero e SQL gratuito pausado deixam a primeira chamada lenta	Médio	Timeout de 30 s no BFF, novas tentativas no ms-pedidos e no front. No dia da apresentação, subir com `-MinReplicas 1` e "aquecer" antes.
R2	Dois pedidos simultâneos do último item podem ser aceitos (a baixa é assíncrona)	Baixo	O estoque não fica negativo: o segundo evento é recusado e registrado. Evolução: reserva de estoque ou saga de compensação que cancela o pedido.
R3	Não há autenticação de usuário; a chave do gateway é visível no front	Médio	Aceitável para o escopo. Evolução: login com Entra ID / OAuth e validação de JWT no gateway.
R4	Cota mensal do SQL gratuito	Baixo	Relay sem consulta periódica; painel pausa a atualização automática com a aba oculta; banco em AutoPause (nunca cobra).
R5	`/aggregated-data` traz no máximo 100 produtos e 100 pedidos	Baixo	Suficiente para a demonstração. Evolução: indicadores calculados no banco ou por projeção alimentada pelos eventos.
R6	Relay do outbox roda dentro do ms-pedidos (uma réplica)	Baixo	Limitamos a 1 réplica. Evolução: bloqueio de linha (`READPAST`) ou relay separado.
R7	Excluir um produto não verifica pedidos antigos que o citam	Baixo	O pedido guarda nome e preço da época, então continua íntegro. Evolução: inativar em vez de excluir.
R8	Mensagens na fila *poison* não têm tratamento automático	Baixo	Ficam guardadas para análise manual. Evolução: alerta e tela de reprocessamento.

12. Glossário

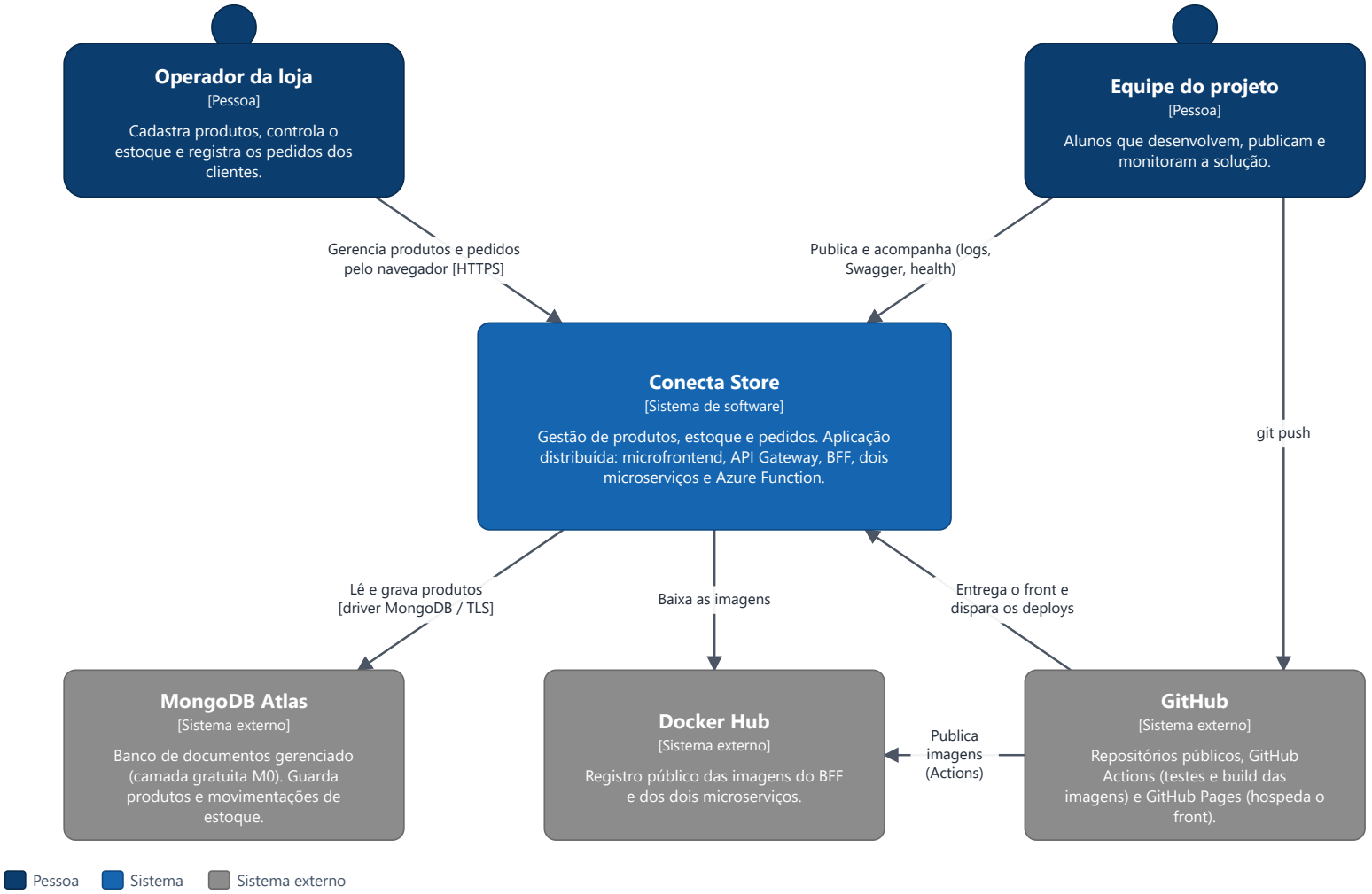
Termo	Significado no projeto
BFF	*Backend for Frontend*: API feita sob medida para a interface, que junta e adapta os dados dos microserviços.
Microfrontend	Interface dividida em partes com build e publicação independentes, montadas no navegador.
Shell / remote	No Module Federation, o *shell* é a aplicação que hospeda; os *remotes* são as partes carregadas por ela.
API Gateway	Ponto único de entrada das APIs: autentica, limita e roteia.
Clean Architecture	Organização em camadas em que as regras de negócio não dependem de framework, banco ou interface.
Vertical Slice	Organização do código por funcionalidade (uma pasta por caso de uso), e não por tipo de arquivo.

Porta / adaptador	Porta é a interface que a aplicação define; adaptador é a implementação concreta (MongoDB, SQL, HTTP).
Composition root	Lugar único onde as implementações são criadas e ligadas (`main.ts`).
Transactional Outbox	Padrão que grava o evento no mesmo banco e transação do dado de negócio, para publicar depois com segurança.
Relay	Componente que lê o outbox e publica os eventos na fila.
Idempotência	Repetir a mesma operação não muda o resultado além da primeira vez.
Consistência eventual	Os dados de serviços diferentes ficam coerentes depois de um curto intervalo, não no mesmo instante.
Correlation-id	Identificador que acompanha uma requisição por todos os serviços, para juntar os logs.
Subscription key	Chave que o API Management exige em cada chamada.
Fila *poison*	Fila para onde vão as mensagens que falharam em todas as tentativas.
AsyncAPI	Especificação para documentar eventos e mensagens, equivalente ao OpenAPI das APIs HTTP.
SKU	Código único do produto.

Parte II — C4 Model · Nível 1: Contexto

C4 — Nível 1: Diagrama de Contexto

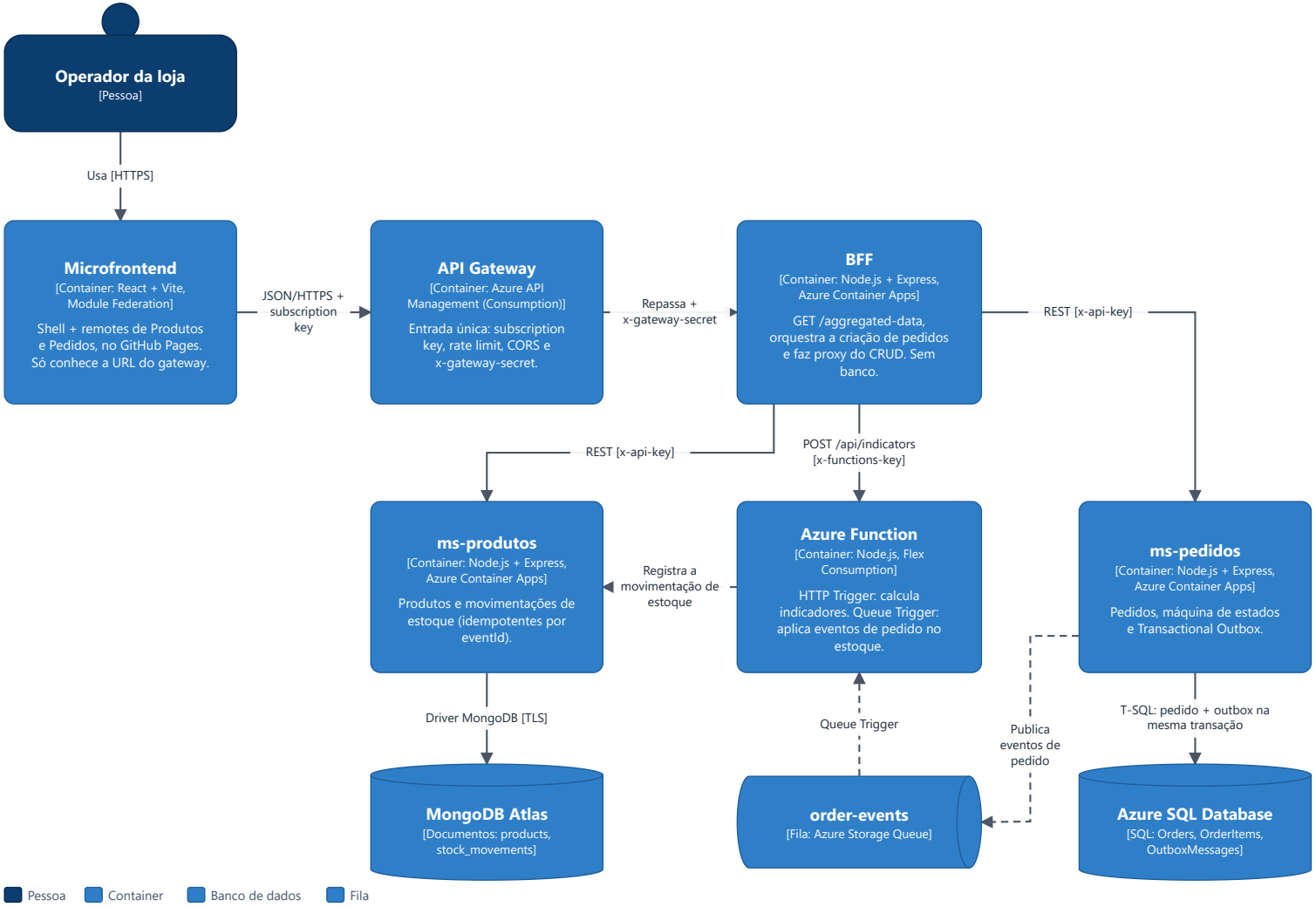
Conecta Store: quem usa o sistema e com quais sistemas externos ele conversa



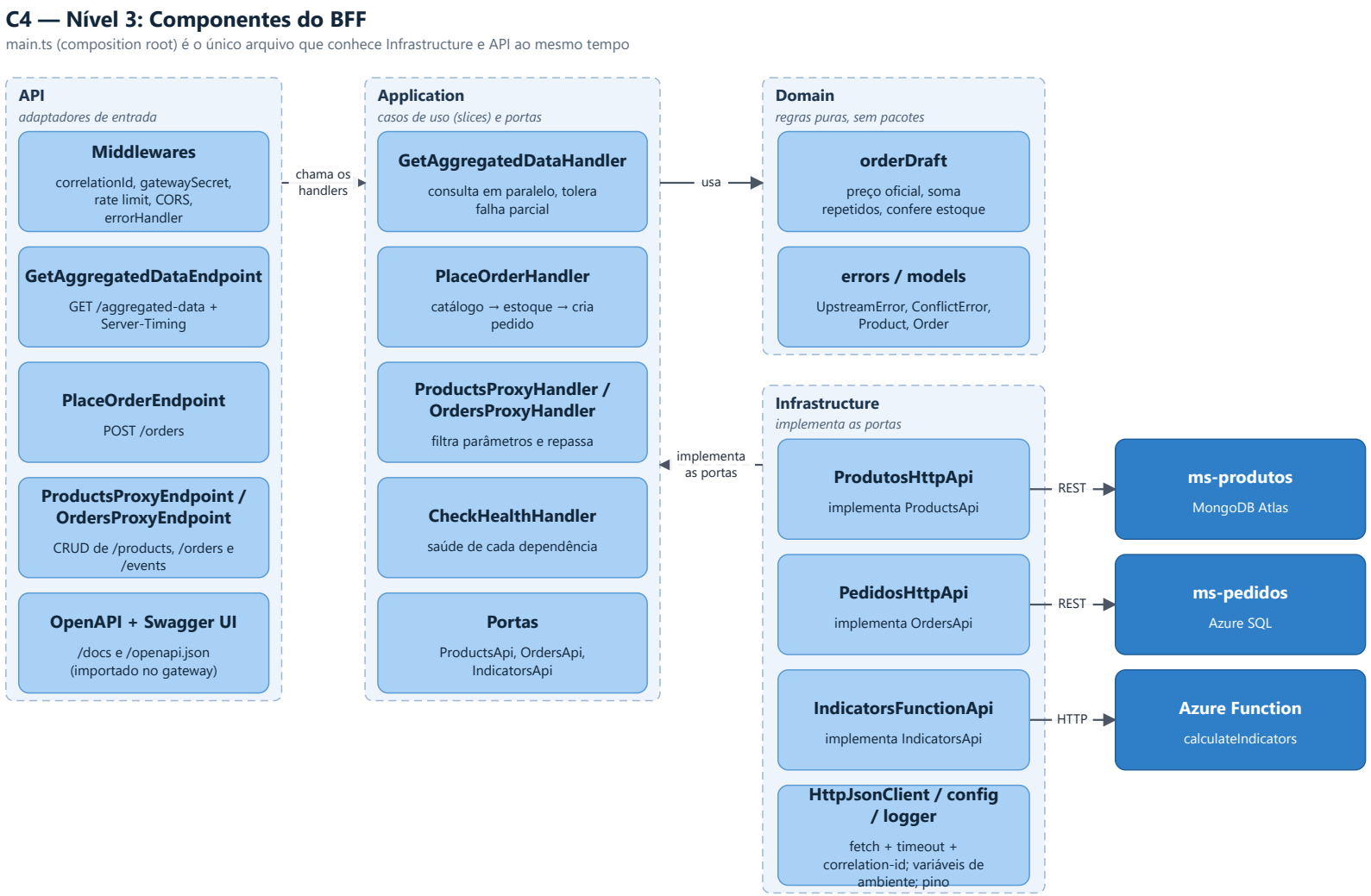
C4 Model · Nível 2: Containers

C4 — Nível 2: Diagrama de Containers

Fluxo principal: Microfrontend → API Gateway → BFF → (ms-produtos, ms-pedidos, Azure Function). Linha tracejada = assíncrono (eventos)



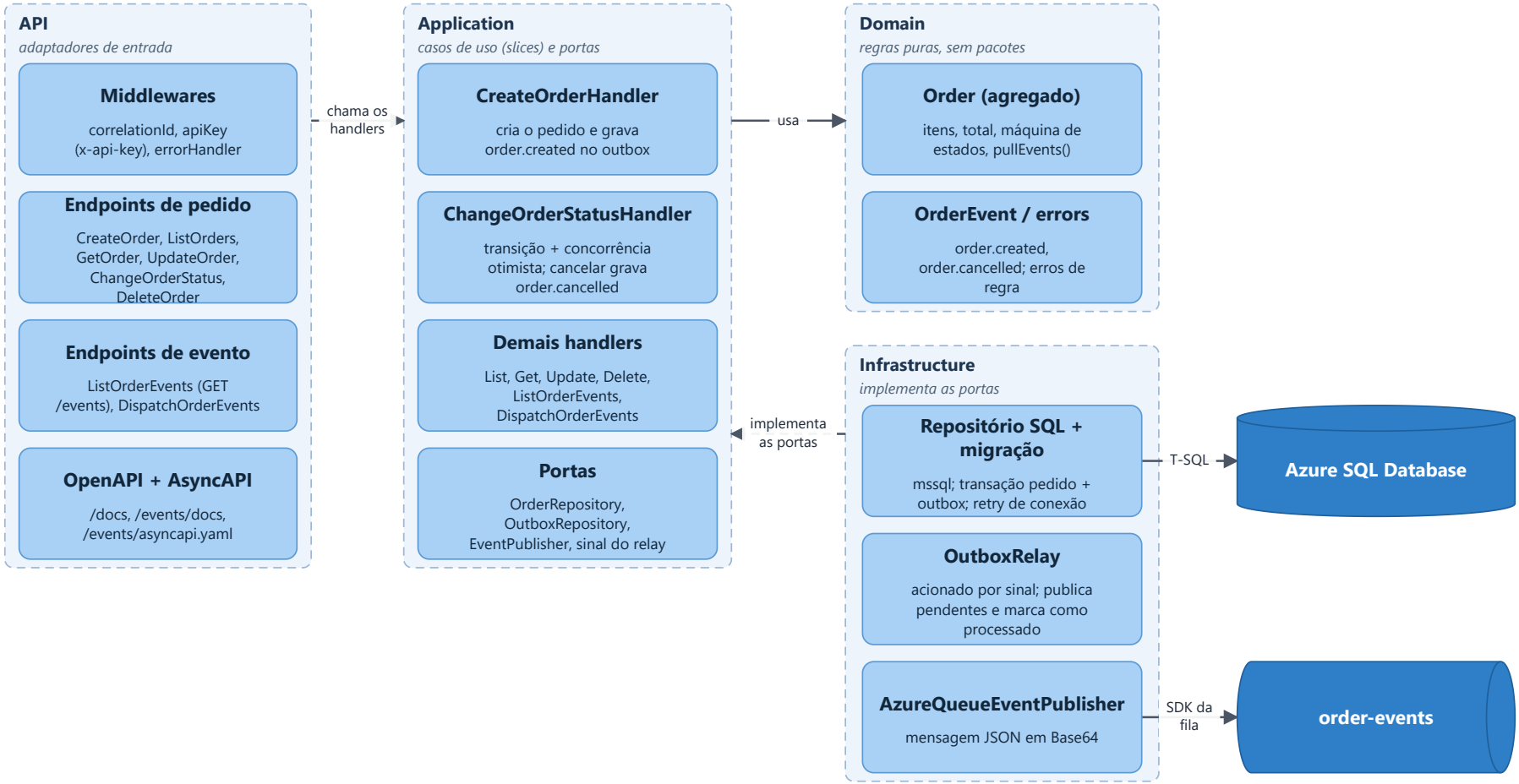
C4 Model · Nível 3: Componentes do BFF



C4 Model · Nível 3: Componentes do ms-pedidos

C4 — Nível 3: Componentes do ms-pedidos

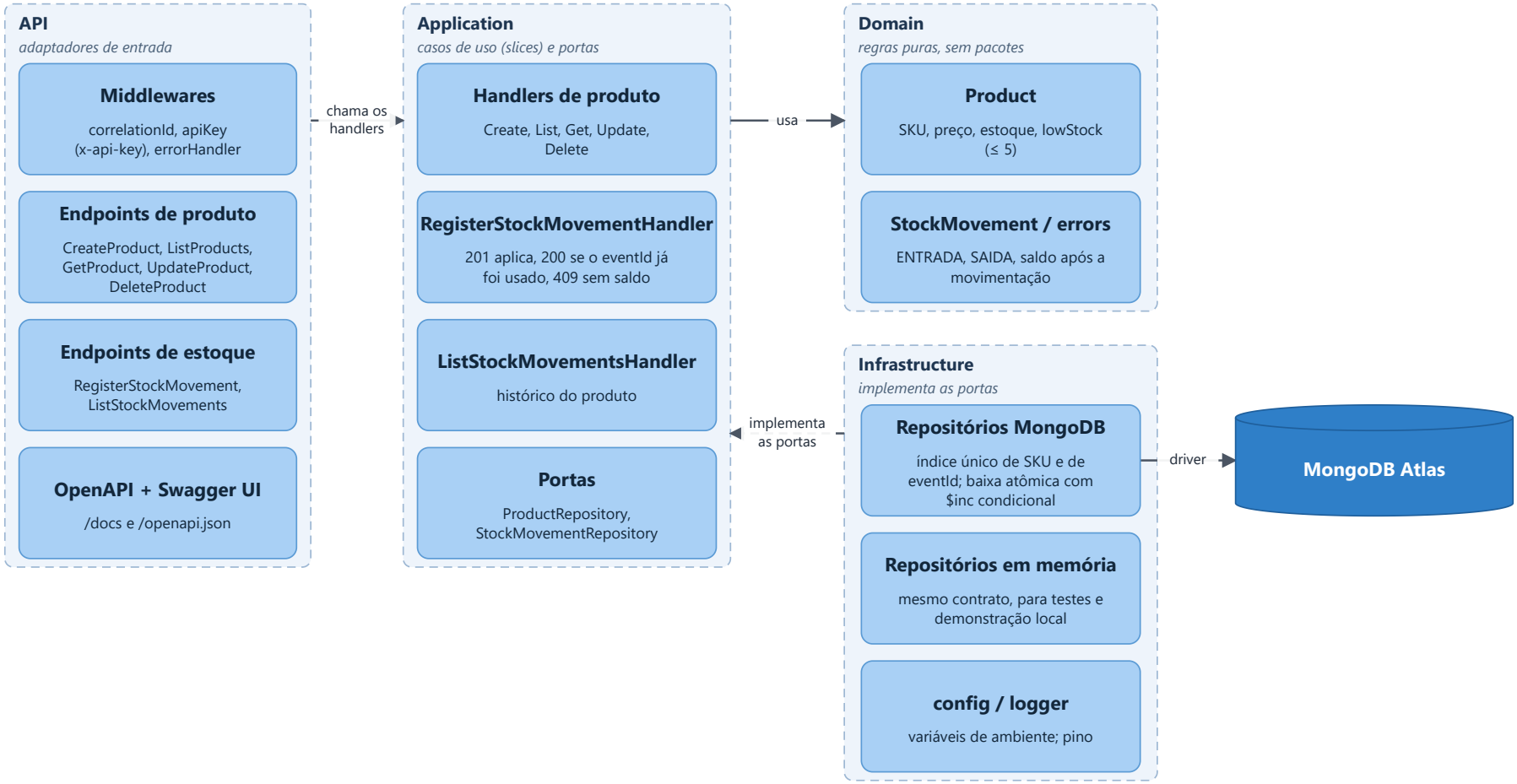
Pedido e evento são gravados na mesma transação (Transactional Outbox); o relay publica na fila



C4 Model · Nível 3: Componentes do ms-produtos

C4 — Nível 3: Componentes do ms-produtos

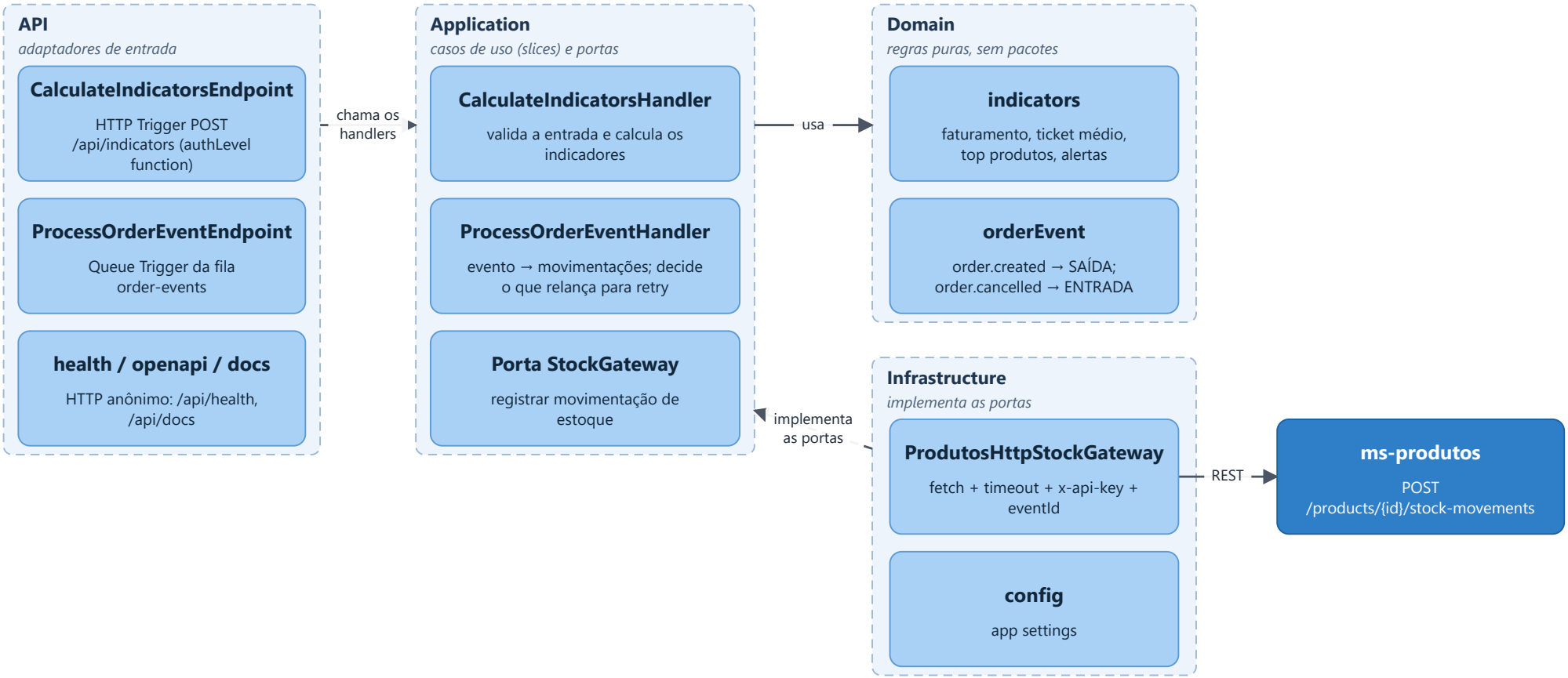
O estoque só muda por movimentação; cada movimentação pode levar um eventId para ser idempotente



C4 Model · Nível 3: Componentes da Azure Function

C4 — Nível 3: Componentes da Azure Function

Mesma separação em camadas dos serviços; os gatilhos (HTTP e fila) são os adaptadores de entrada



Os diagramas são gerados a partir de `docs/diagramas/definicoes.mjs` (repositório pjbl-arquitetura). Os arquivos SVG ficam em `docs/c4/` e o modelo em Structurizr DSL em `docs/c4/workspace.dsl`.

Parte III — Canvas do projeto

Objetivo

- Construir e publicar uma aplicação distribuída completa (microfrontend, gateway, BFF, dois microserviços, Function) para gestão de produtos e pedidos.
- Aplicar Clean Architecture, Vertical Slice e arquitetura orientada a eventos, com evidência por testes.

Justificativa

- Praticar, num caso pequeno e real, os padrões vistos na disciplina.
- Produtos × pedidos obriga a integrar dois domínios, o que justifica BFF e eventos.

Partes interessadas

- Professor (avaliador)
- Equipe do projeto
- Operador da loja (usuário)

Requisitos

- CRUD de produtos e de pedidos
- `GET /aggregated-data`
- Eventos de pedido
- Swagger em tudo
- Gateway na entrada
- Testes unitários e de arquitetura

Entregas

- 6 repositórios públicos
- 3 imagens no Docker Hub
- Aplicação na nuvem
- PDF (ARC42, C4, Canvas, prints)
- Vídeo no YouTube

Premissas

- Contas gratuitas disponíveis (Azure for Students, Atlas, GitHub, Docker Hub)
- Volume baixo de dados
- Um único tipo de usuário

Restrições

- Custo zero
- BFF em Node.js
- MongoDB Atlas e Azure SQL gratuitos
- Demonstração só com URLs públicas
- Nenhum segredo no Git

Riscos

- Cold start e banco pausado
- Cota das camadas gratuitas
- Consistência eventual do estoque
- Região indisponível na assinatura de estudante

Solução (arquitetura)

- React + Module Federation → Azure API Management → BFF (Express) → ms-produtos (MongoDB Atlas), ms-pedidos (Azure SQL) e Azure Function.
- Outbox → Storage Queue → Function → estoque, com idempotência.
- TypeScript em tudo; GitHub Actions; Container Apps.

Equipe

- Arthur Ambrozewicz Cidral
- Erick Marlon Barbieri da Silva
- Luan Alberti Estevinho
- Vinícius de Oliveira Garcia

Linha do tempo

- Entrega 1: proposta e contexto
- Entrega 2: modelagem e C4 inicial
- Entrega 3: implementação, nuvem, documentação e vídeo

Custos

- GitHub, Docker Hub, Atlas M0: gratuito
- Azure SQL: oferta gratuita
- APIM, Functions, Container Apps: dentro da franquia de consumo
- **Total previsto: R$ 0**

Critérios de sucesso

- Fluxo completo demonstrado na nuvem: produto → pedido → evento → estoque → painel.
- Todos os testes passando na esteira, inclusive os de arquitetura.
- Todos os itens do enunciado conferidos (Parte V).

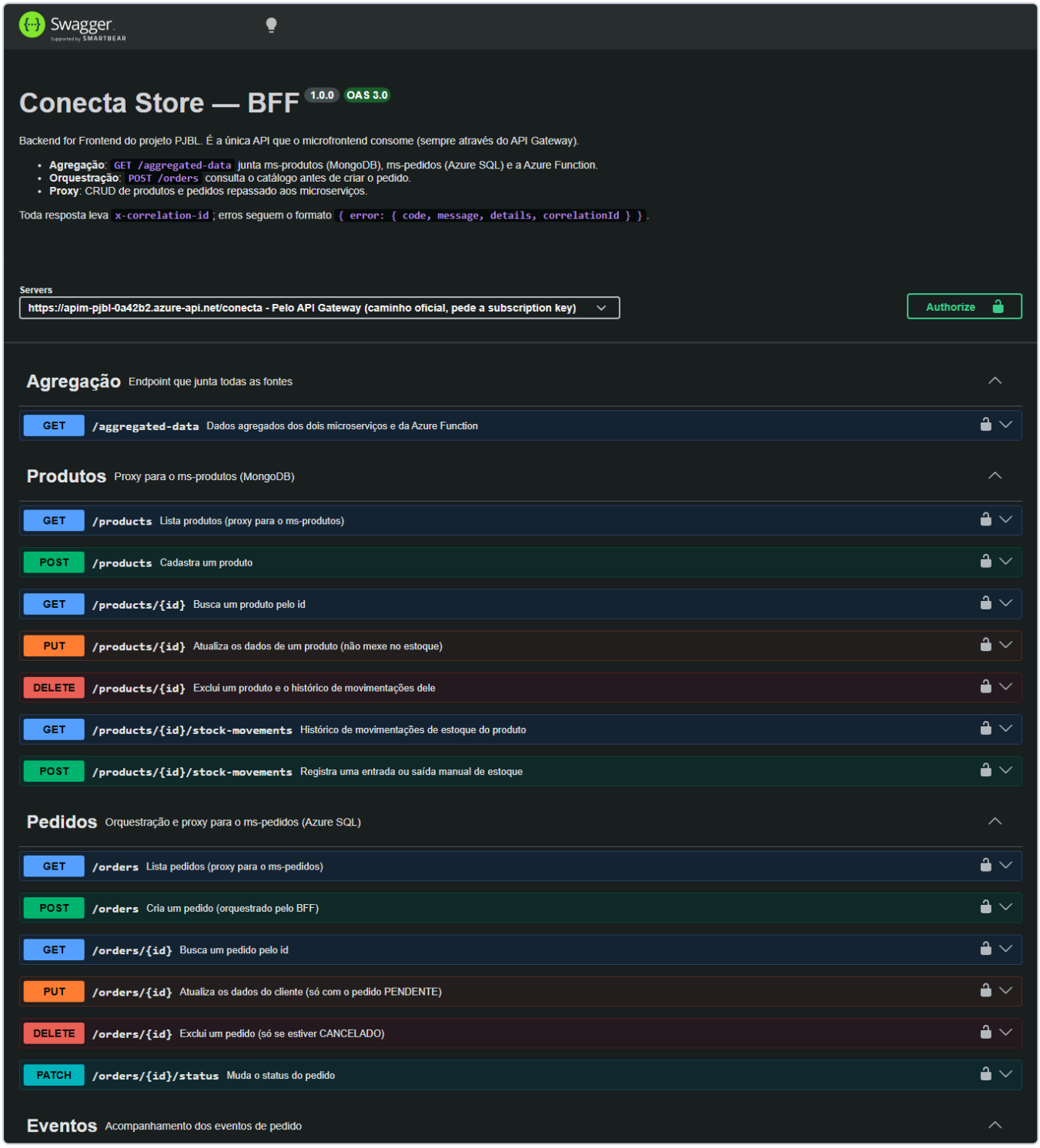
Fora do escopo

- Login e perfis de usuário
- Pagamento e emissão de nota
- Alta disponibilidade e múltiplas regiões

Parte IV — Prints e evidências

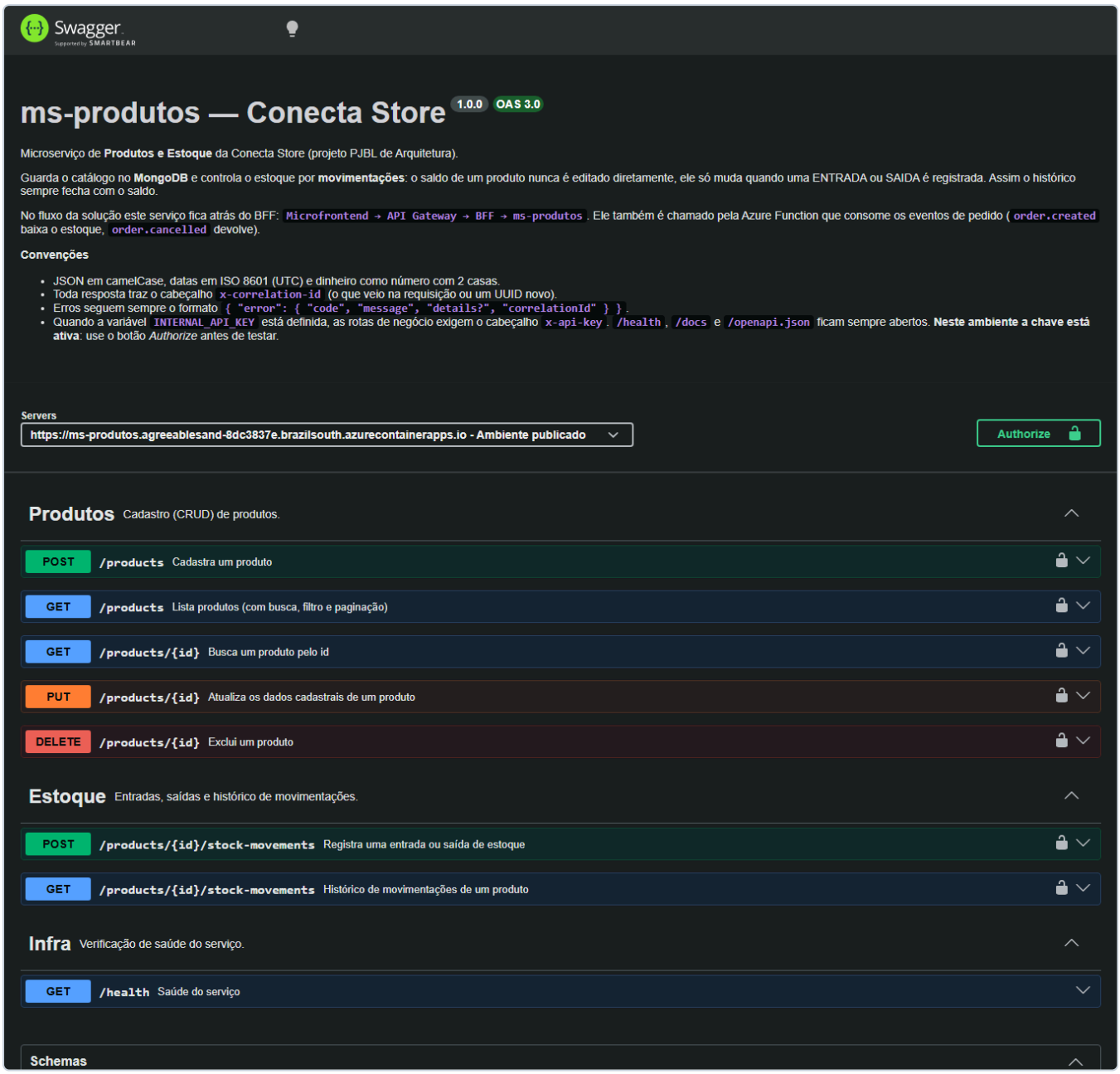
As capturas de páginas foram feitas nas URLs públicas listadas no início do documento. As duas últimas são saídas de terminal (testes), que rodam na máquina do desenvolvedor e no GitHub Actions.

Swagger do BFF

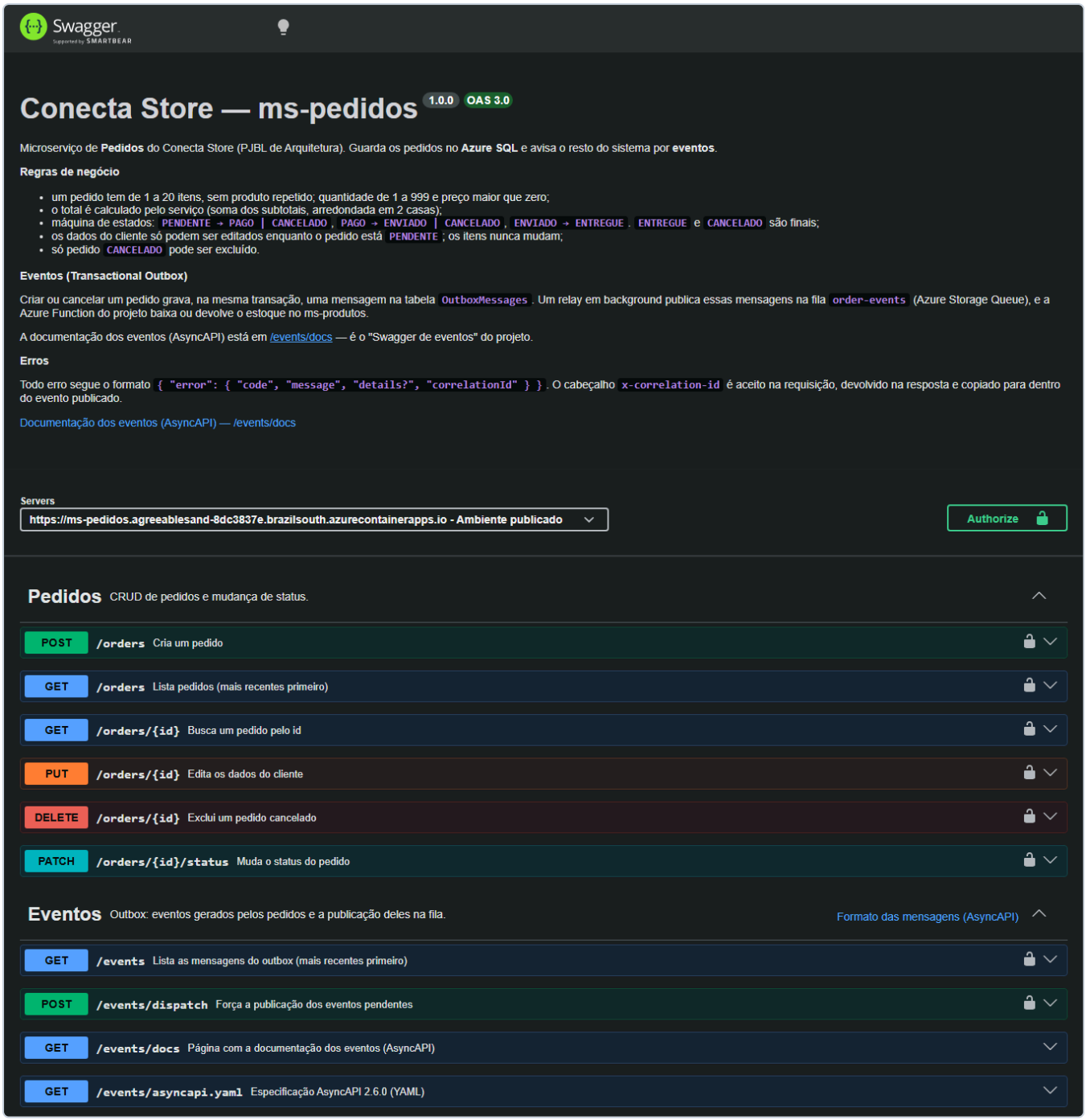


Swagger UI do BFF, com o endpoint obrigatório GET /aggregated-data

Swagger dos microserviços



Swagger UI do ms-produtos (MongoDB Atlas)

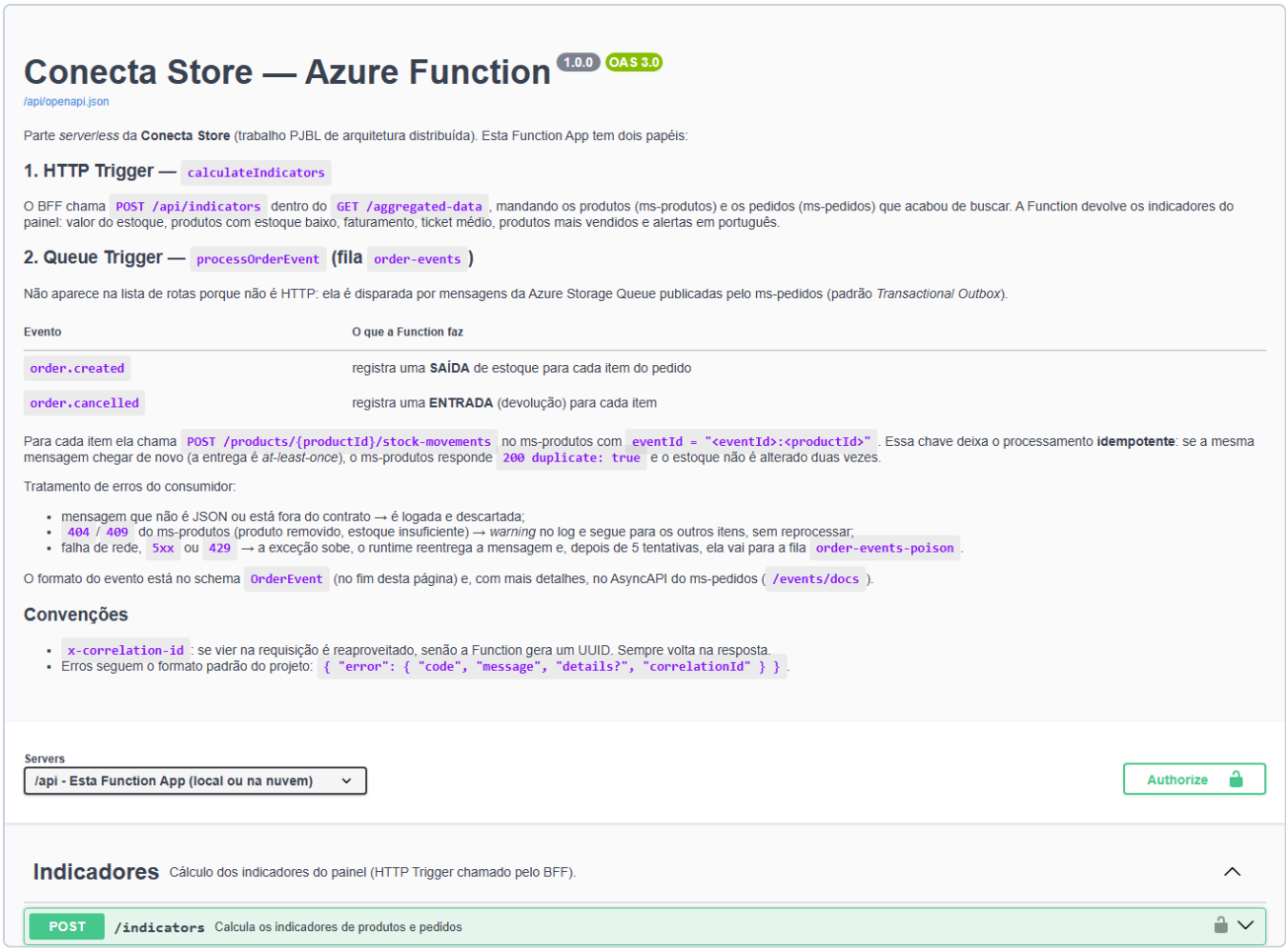


Swagger UI do ms-pedidos (Azure SQL)

Eventos e Azure Function

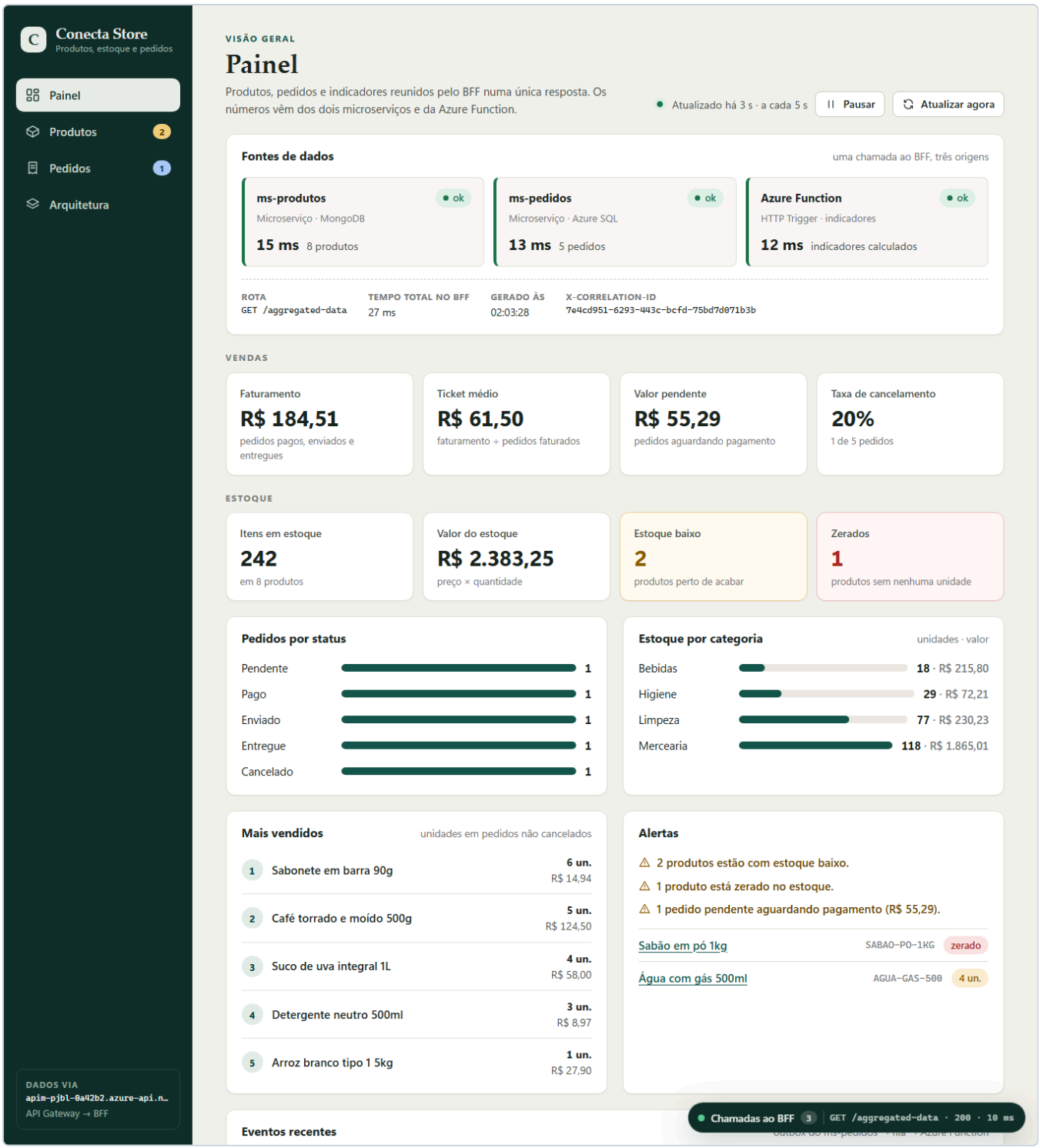


Documentação AsyncAPI dos eventos order.created e order.cancelled

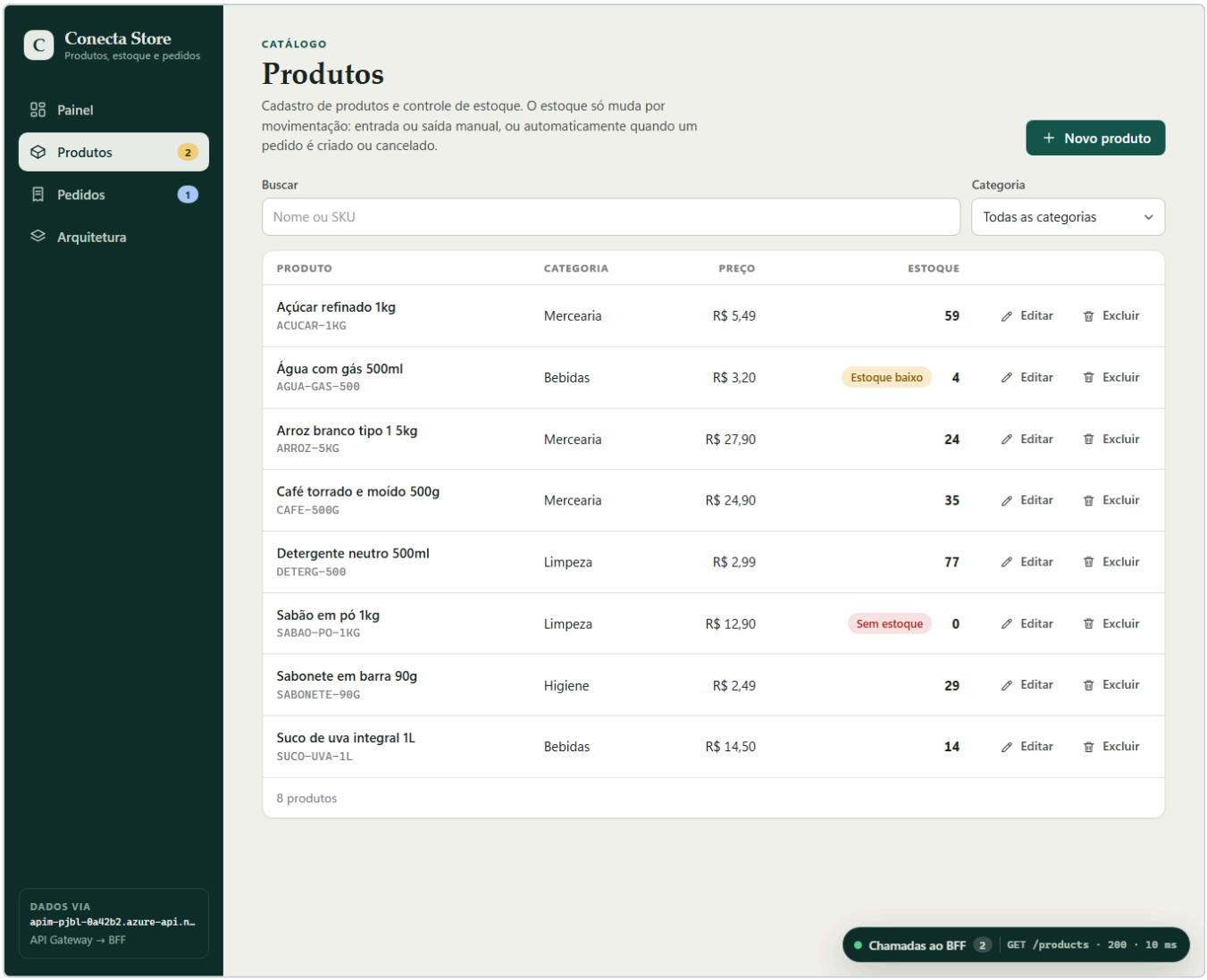


Swagger UI da Azure Function (HTTP Trigger calculateIndicators)

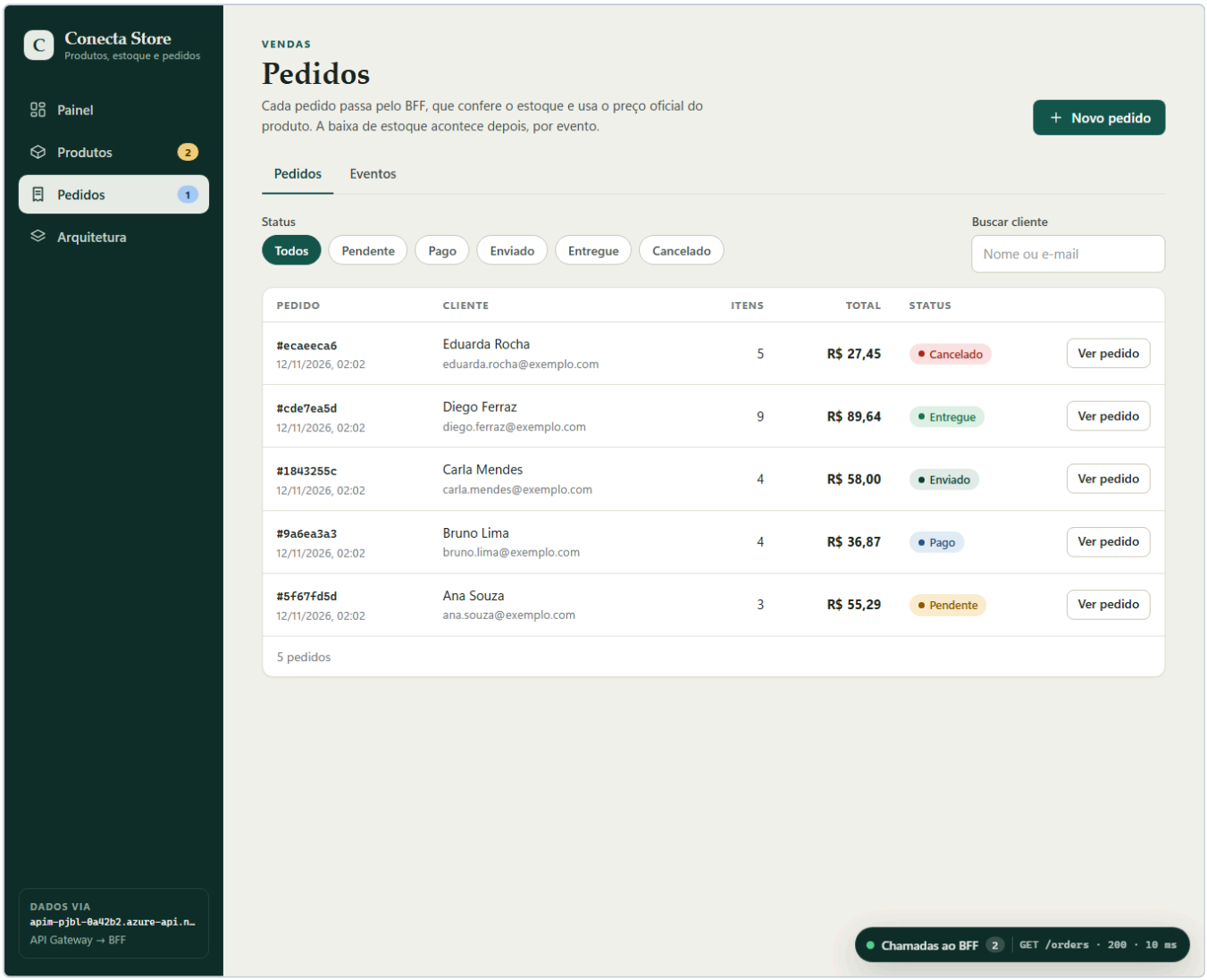
Frontend funcionando



Painel: indicadores calculados pela Function, situação de cada fonte e eventos recentes

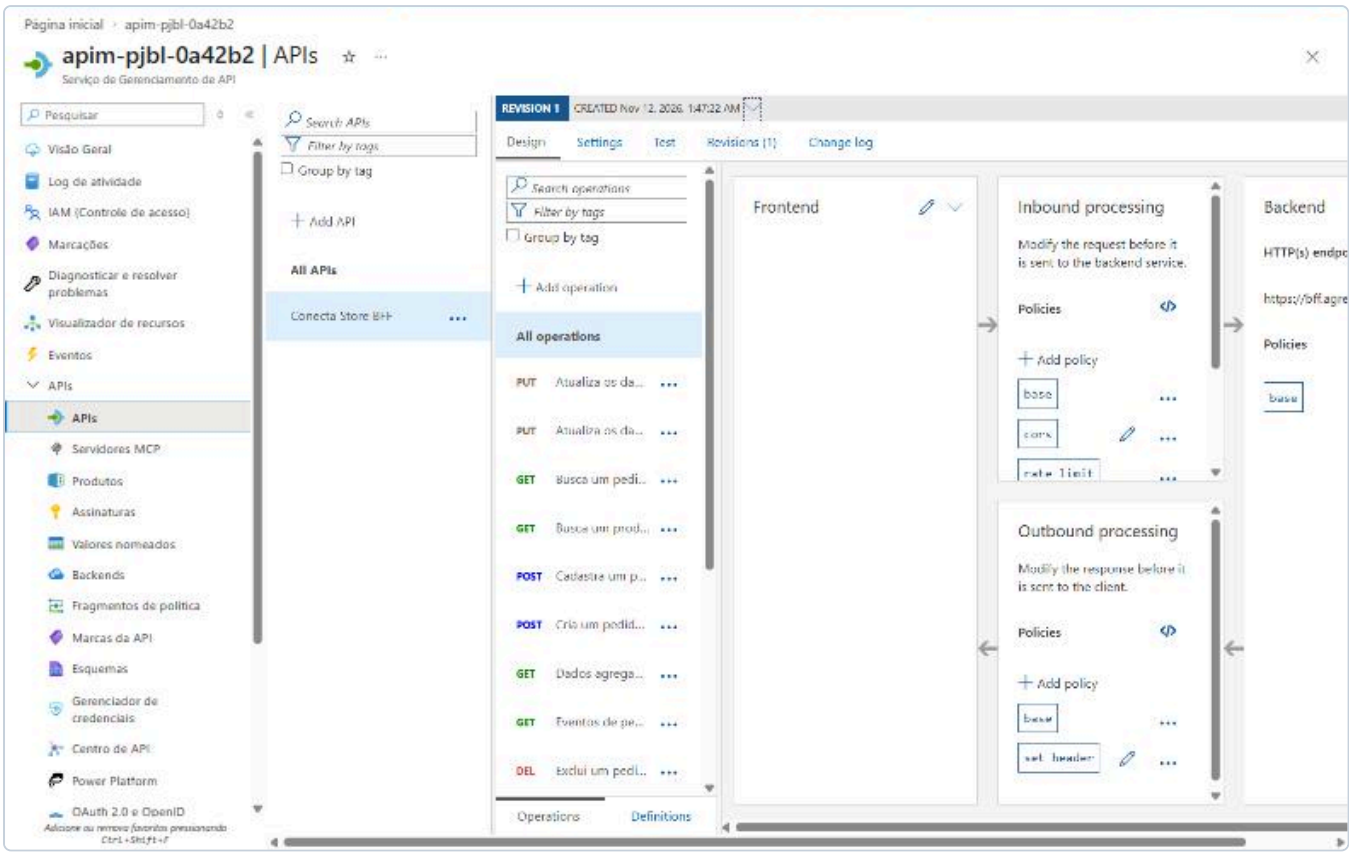


Microfrontend de Produtos: lista, estoque baixo e movimentações



Microfrontend de Pedidos: lista, status e eventos

API Gateway



API importada no Azure API Management, com a policy aplicada

API Gateway — roteamento, chave obrigatória e bloqueio de acesso direto ao BFF

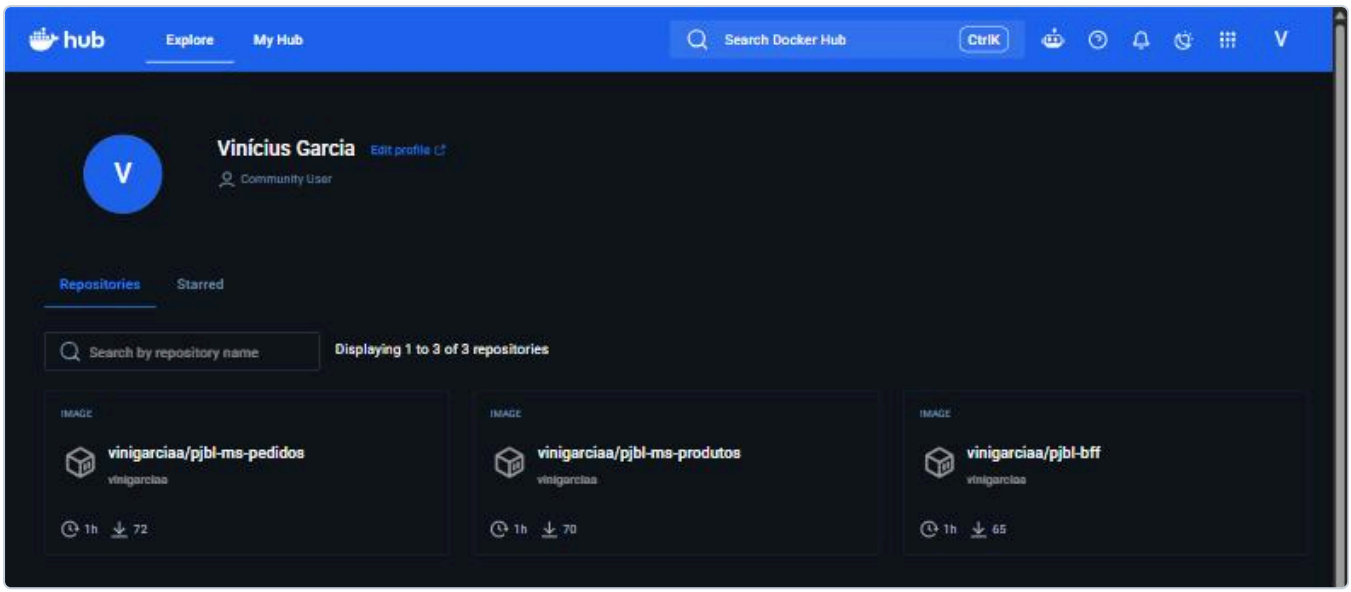
```
$ curl -i https://apim-pjbl-0a42b2.azure-api.net/conecta/aggregated-data                    # sem chave
HTTP 401 Access Denied
  { "statusCode": 401, "message": "Access denied due to missing subscription key. Make sure to include subscription key when making requests to an API." }

$ curl -i -H "Ocp-Apim-Subscription-Key: ****" https://apim-pjbl-0a42b2.azure-api.net/conecta/aggregated-data
HTTP 200 OK
  x-gateway: azure-api-management
  x-served-by: pjbl-bff
  x-correlation-id: 6241de37-e170-495d-b182-86ec3a849d83
  server-timing: ms-produtos;dur=16,ms-pedidos;dur=14,azure-function;dur=12,total;dur=28
  {"generatedAt":"2026-11-12T05:02:41.940Z","correlationId":"6241de37-e170-495d-b182-86ec3a849d83","partial":false,"totalDurationMs":28,"products":{"items":
[{"id":"fd03ac57-b891-4381-9baf-763571f38f16","sku":"ACUCAR-1KG","name":"Açú…

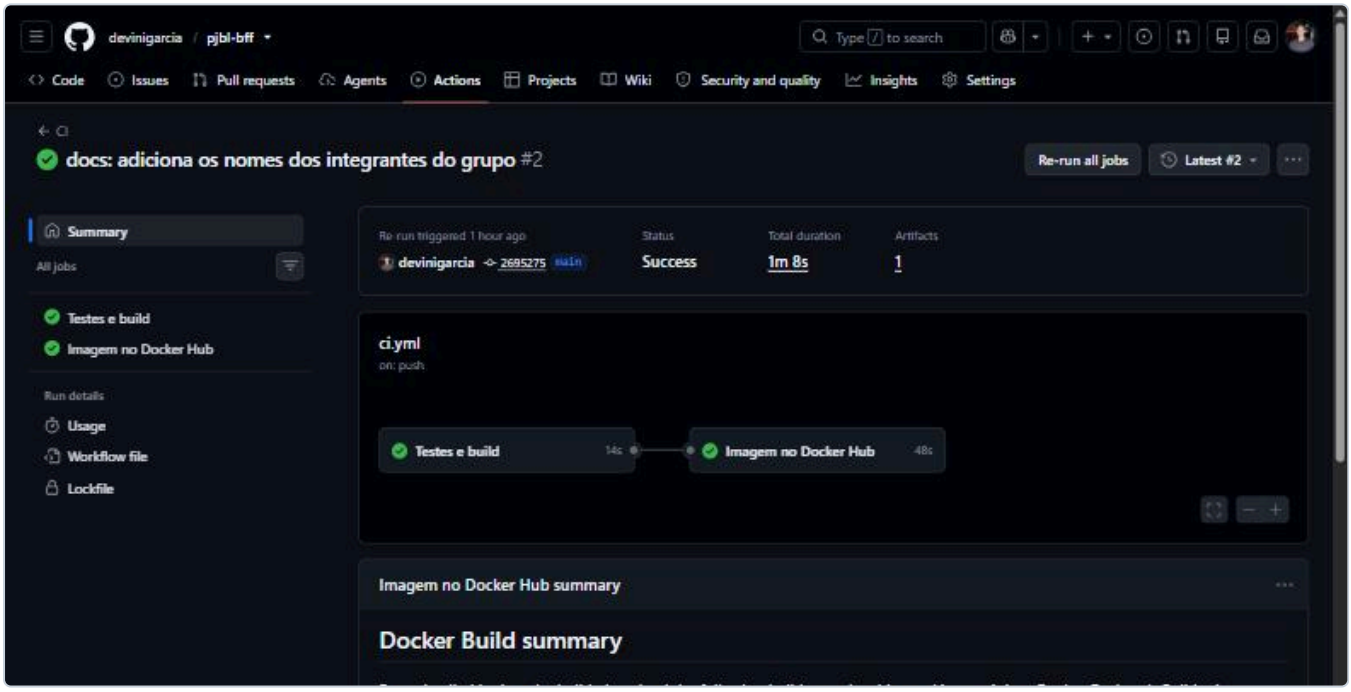
$ curl -i https://bff.agreeablesand-8dc3837e.brazilsouth.azurecontainerapps.io/aggregated-data                    # direto no BFF, pulando o gateway
HTTP 401 Unauthorized
  x-served-by: pjbl-bff
  x-correlation-id: 9f4c580b-3555-4424-817b-a6b4b0e99f8f
  {"error":{"code":"UNAUTHORIZED","message":"Acesso permitido somente pelo API Gateway.","correlationId":"9f4c580b-3555-4424-817b-a6b4b0e99f8f"}}
```

Chamadas ao gateway: 401 sem chave, 200 com chave, e 401 direto no BFF

Docker Hub, GitHub Actions e testes



Imagens publicadas no Docker Hub



Esteira do GitHub Actions concluída com sucesso

Testes dos cinco projetos (unitários, integração e arquitetura)

```
$ node scripts/resumo-testes.mjs
✓ pjbl-ms-produtos
    arquivos: 14 passed (14)
    testes:   161 passed (161)
✓ pjbl-ms-pedidos
    arquivos: 11 passed | 1 skipped (12)
    testes:   210 passed | 18 skipped (228)
✓ pjbl-function
    arquivos: 10 passed (10)
    testes:   85 passed (85)
✓ pjbl-bff
    arquivos: 6 passed (6)
    testes:   96 passed (96)
✓ pjbl-microfrontend
    arquivos: 14 passed (14)
    testes:   89 passed (89)

✓ tudo passou — 641 testes passed nos 5 projetos
```

Testes passando, incluindo os de arquitetura

Teste de arquitetura com uma violação plantada — pjbl-bff

```
$ node scripts/demo-violacao.mjs
Plantando em src/domain/orderDraft.ts:
  import express from 'express'; // violação plantada de propósito

   x Clean Architecture — direção das dependências > Domain não depende de nenhuma outra camada nem de pacotes externos 17ms
   ✓ Clean Architecture — direção das dependências > API não importa a Infrastructure (a ligação é feita só no main.ts) 0ms
   x Clean Architecture — direção das dependências > Express e demais pacotes web só aparecem na camada API 2ms
   ✓ Vertical Slice — organização por feature > uma feature não importa código de outra feature 0ms
   x as regras detectam violações plantadas > pega o Domain importando um pacote ou outra camada 5ms
   ✓ as regras detectam violações plantadas > pega a Application importando Infrastructure, API ou um framework 2ms
   ✓ as regras detectam violações plantadas > pega a API importando a Infrastructure e a Infrastructure importando a API 1ms
   ✓ as regras detectam violações plantadas > pega uma feature importando outra 1ms
   x as regras detectam violações plantadas > pega fetch fora da Infrastructure e Express fora da API 3ms
 Test Files  1 failed (1)
      Tests  4 failed | 17 passed (21)
⎯⎯⎯⎯⎯⎯ Failed Tests 4 ⎯⎯⎯⎯⎯⎯
 FAIL  tests/architecture/architecture.test.ts > Clean Architecture — direção das dependências > Domain não depende de nenhuma outra camada nem de
pacotes externos
AssertionError: expected [ Array(1) ] to deeply equal []
- Expected
+ Received
- []
+ [
+   "domain/orderDraft.ts importa \"express\"",
+ ]
 FAIL  tests/architecture/architecture.test.ts > Clean Architecture — direção das dependências > Express e demais pacotes web só aparecem na camada
API
AssertionError: expected [ Array(1) ] to deeply equal []
- Expected
+ Received
- []
+ [
+   "domain/orderDraft.ts importa \"express\"",
+ ]
 FAIL  tests/architecture/architecture.test.ts > as regras detectam violações plantadas > pega o Domain importando um pacote ou outra camada
AssertionError: expected [ …(2) ] to have a length of 1 but got 2
- Expected
+ Received
- 1
+ 2
     87|   it('pega o Domain importando um pacote ou outra camada', () => {
 FAIL  tests/architecture/architecture.test.ts > as regras detectam violações plantadas > pega fetch fora da Infrastructure e Express fora da API
AssertionError: expected [ …(2) ] to have a length of 1 but got 2
- Expected
+ Received
- 1

código de saída: 1 (a esteira barraria esse commit)

Arquivo restaurado ao estado original.
```

Teste de arquitetura acusando uma violação plantada de propósito

Parte V — Conferência com o enunciado

Item pedido		Onde está
Microfrontend (SPA React) que consome o BFF	✓	pjbl-microfrontend · https://devinigarcia.github.io/pjbl-microfrontend
BFF em Node.js: agregação, proxy CRUD, chamada à Function	✓	pjbl-bff · seção 5 e C4 nível 3
Endpoint `GET /aggregated-data` com resposta única	✓	seção 6.1 · Swagger do BFF
Microserviço 1 com MongoDB Atlas, CRUD, Swagger	✓	pjbl-ms-produtos
Microserviço 2 com Azure SQL, CRUD, Swagger	✓	pjbl-ms-pedidos
Azure Function HTTP Trigger / Message Trigger	✓	pjbl-function (os dois gatilhos)
API Gateway: roteamento, segurança, entrada central	✓	pjbl-arquitetura/gateway · seção 8.3 · ADR-05
Clean Architecture (Domain, Application, Infrastructure, API)	✓	seções 5.2 e 8.1 · todos os backends
Vertical Slice (`CreateOrder`, `GetOrder`...)	✓	seção 5.2 · pastas `features/`
Event-Driven Architecture	✓	seções 6.2 e 8.2 · AsyncAPI
Integração com múltiplos bancos	✓	MongoDB Atlas + Azure SQL
Testes unitários e de arquitetura	✓	seção 8.6 · 659 testes
Imagens no Docker Hub (microserviços + BFF)	✓	página de links
Repositórios públicos no GitHub	✓	página de links (5 pedidos + 1 de arquitetura)
README em cada projeto (arquitetura, tecnologias, como rodar, alunos)	✓	raiz de cada repositório
ARC42 com todos os links	✓	Parte I e página de links
C4 Model (Contexto, Containers, Componentes)	✓	Parte II
Canvas	✓	Parte III
Prints: Swagger BFF, Swagger microserviços, Frontend, API Gateway	✓	Parte IV
Vídeo no YouTube (não listado), com todos do grupo	✓	pendente: URL_VIDEO